Chương 1

GIỚI THIỆU

Chương 1 cung cấp một cái nhìn tổng quan về lĩnh vực Trí tuệ nhân tạo (AI), bắt đầu bằng việc giới thiệu chung về AI và các cách tiếp cận chính. Chương này sẽ đưa người đọc đi sâu vào lịch sử phát triển của AI, từ những khái niệm ban đầu đến những bước tiến đột phá. Đồng thời cho người đọc thấy được vị trí của học sâu trong lĩnh vực AI. Phần cuối chương giới thiệu về Pytorch, một framework phổ biến, bao gồm các khái niệm về Tensors, Indexing, tự động tính đạo hàm, Mạng nơ-ron và các lớp module khác.

1.1 Tổng quan về Trí tuệ nhân tạo

1.1.1 Giới thiệu

Vào cuối năm 2017, một cuộc cách mạng lặng lẽ đã diễn ra. AlphaZero, một chương trình trí tuệ nhân tạo (AI) do Google DeepMind phát triển, đã đánh bại Stockfish - chương trình cờ vua mạnh nhất thế giới lúc bấy giờ. Chiến thắng của AlphaZero rất thuyết phục: nó thắng 28 ván, hòa 72, và không thua ván nào. Năm sau, nó tiếp tục khẳng định sự vượt trội: trong 1.000 ván đấu với Stockfish, nó thắng 155, thua 6, và hòa số còn lại.

Thông thường, việc một chương trình cờ vua đánh bại chương trình khác chỉ quan trọng với một số ít người đam mê. Nhưng AlphaZero không phải là một chương trình cờ vua thông thường. Các chương trình trước đây dựa vào những nước đi được con người nghĩ ra, thực hiện, và tải lên - nói cách khác, chúng phụ thuộc vào kinh nghiệm, kiến thức và chiến lược của con người. Ưu điểm chính của các chương trình này so với con người không phải là sự sáng tạo mà là sức mạnh xử lý vượt trội, cho phép chúng đánh giá nhiều tùy chọn hơn trong một khoảng thời gian nhất định. Ngược lại, AlphaZero không có bất kỳ nước đi, chuỗi kết hợp, hay chiến lược nào được lập trình sẵn từ con người. Phong cách của AlphaZero hoàn toàn là sản phẩm của quá trình huấn luyện AI: những người tạo ra nó chỉ cung cấp luật chơi cờ vua và hướng dẫn nó phát triển chiến lược để tối đa hóa tỷ lệ thắng so với thua. Chỉ sau bốn giờ huấn luyện bằng cách tự chơi với chính mình, AlphaZero đã trở thành chương trình cờ vua hiệu quả nhất thế giới. Tính đến thời điểm này, chưa có con người nào đánh bại được nó.

Các chiến thuật mà AlphaZero triển khai rất khác thường, thậm chí mang tính sáng tạo. Nó sẵn sàng hy sinh những quân cờ mà con người cho là thiết yếu, bao gồm cả quân hậu. Nó thực hiện những nước đi mà con người không dạy nó cân nhắc, và trong nhiều trường hợp, con người chưa từng nghĩ đến. AlphaZero áp dụng những chiến thuật bất ngờ như vậy vì, sau khi tự chơi nhiều ván, nó dự đoán rằng những nước đi đó sẽ tối đa hóa khả năng chiến thắng của nó. AlphaZero không có chiến lược theo nghĩa của con người (mặc dù phong cách chơi của nó đã thúc đẩy nghiên cứu thêm của con người về trò chơi này). Thay vào đó, nó có một logic riêng, được hình thành bởi khả năng nhận biết các mẫu nước đi trong vô số các khả năng mà trí óc con người không thể hoàn toàn nắm bắt hoặc sử dụng. Ở mỗi giai đoạn của ván cờ, AlphaZero đánh giá sự sắp xếp của các quân cờ dựa trên những gì nó đã học được từ các mẫu nước cờ và chọn nước đi mà nó cho rằng có khả năng dẫn đến chiến thắng cao nhất. Sau khi quan sát và phân tích lối chơi của AlphaZero, Garry Kasparov, đại kiện tướng và nhà vô địch thế giới, đã tuyên bố: "Cờ vua đã bị rung chuyển tận gốc rễ bởi AlphaZero." Khi AI thăm dò các giới hạn của trò chơi mà họ đã dành cả đời để làm chủ, những kỳ thủ giỏi nhất thế giới chỉ có thể làm một điều: quan sát và học hỏi.

Đầu năm 2020, các nhà nghiên cứu tại Viện Công nghệ Massachusetts (MIT) đã công bố phát hiện ra một loại kháng sinh mới có khả năng tiêu diệt các chủng vi khuẩn cho đến lúc đó vẫn kháng lại mọi loại kháng sinh đã biết. Các nỗ lực nghiên cứu và phát triển tiêu chuẩn để tạo ra một loại thuốc mới thường mất nhiều năm làm việc tốn kém và đầy khó khăn, khi các nhà nghiên cứu bắt đầu với hàng nghìn phân tử tiềm năng và thông qua quá trình thử nghiệm, loại bỏ dần để chỉ còn lại một số ít ứng viên khả thi. Thông thường, các nhà nghiên cứu phải đưa ra các phỏng đoán có cơ sở giữa hàng nghìn phân tử hoặc

các chuyên gia thử nghiệm với các phân tử đã biết, hy vọng tìm được kết quả khả quan bằng cách điều chỉnh cấu trúc phân tử của một loại thuốc hiện có.

MIT đã làm điều khác biệt: họ mời AI tham gia vào quy trình. Đầu tiên, các nhà nghiên cứu xây dựng một "tập huấn luyện" gồm 2.000 phân tử đã biết. Tập huấn luyện này mã hóa dữ liệu về từng phân tử, từ trọng lượng nguyên tử đến các loại liên kết mà chúng chứa, đến khả năng ức chế sự phát triển của vi khuẩn. Từ tập huấn luyện này, AI đã "học" được các đặc điểm của các phân tử được dự đoán là có khả năng kháng khuẩn. Điều thú vị là, nó xác định được những đặc điểm không được mã hóa cụ thể, thực sự là những đặc điểm vượt qua nhận thức hoặc sự phân loại của con người.

Khi quá trình huấn luyện hoàn tất, các nhà nghiên cứu đã hướng dẫn AI khảo sát một thư viện gồm 61.000 phân tử, các loại thuốc được FDA phê duyệt, và các sản phẩm tự nhiên để tìm các phân tử đáp ứng các tiêu chí: (1) AI dự đoán sẽ có hiệu quả như kháng sinh, (2) không giống bất kỳ loại kháng sinh nào hiện có, và (3) AI dự đoán sẽ không độc hại. Trong số 61.000 phân tử, một phân tử đáp ứng được các tiêu chí. Các nhà nghiên cứu đã đặt tên cho nó là halicin nhằm vinh danh AI HAL trong bộ phim 2001: A Space Odyssey.

Các nhà lãnh đạo dự án tại MIT nhấn mạnh rằng việc tìm ra halicin thông qua các phương pháp nghiên cứu và phát triển truyền thống sẽ "tốn kém đến mức không thể thực hiện", nói cách khác, điều đó sẽ không xảy ra. Thay vào đó, bằng cách huấn luyện một chương trình phần mềm để nhận diện các mẫu cấu trúc trong các phân tử đã chứng minh hiệu quả trong việc chống lại vi khuẩn, quy trình xác định đã trở nên hiệu quả và tiết kiệm hơn. Chương trình không cần hiểu tại sao các phân tử hoạt động. Thực tế, trong một số trường hợp, không ai biết tại sao một số phân tử lại có hiệu quả. Tuy nhiên, AI vẫn có thể quét qua thư viện các ứng viên để xác định một phân tử có khả năng thực hiện một chức năng mong muốn nhưng chưa được khám phá: tiêu diệt một chủng vi khuẩn mà chưa có loại kháng sinh nào biết đến.

Halicin là một thành công lớn. So với cờ vua, lĩnh vực dược phẩm phức tạp hơn rất nhiều. Trong cờ vua, chỉ có sáu loại quân, mỗi loại chỉ di chuyển theo những cách nhất định, và chỉ có một điều kiện chiến thắng duy nhất: bắt được vua của đối thủ. Ngược lại, việc tìm ra một ứng viên thuốc tiềm năng liên quan đến hàng trăm ngàn phân tử có thể tương tác với các chức năng sinh học khác nhau của vi khuẩn và virus theo những cách phức tạp và thường chưa được biết đến. Hãy tưởng tượng một trò chơi có hàng ngàn quân, hàng trăm điều kiện chiến thắng và các quy tắc chỉ được biết một phần. Sau khi nghiên cứu vài ngàn trường hợp thành công, AI đã tạo ra một chiến thắng mới một loại kháng sinh mới mà trước đó chưa ai phát hiện.

Điều hấp dẫn nhất là những gì AI đã phát hiện ra. Các nhà hóa học đã tạo ra các khái niệm như trọng lượng nguyên tử và liên kết hóa học để mô tả đặc điểm của các phân tử. Nhưng AI đã xác định các mối quan hệ mà con người không phát hiện được hoặc có thể thậm chí không thể mô tả được. AI mà các nhà nghiên cứu MIT đã huấn luyện không chỉ tái tạo các kết luận dựa trên các đặc điểm quan sát được trước đây của các phân tử. Thay vào đó, nó phát hiện ra các đặc điểm phân tử mới, các mối quan hệ giữa các yếu tố trong cấu trúc của chúng và khả năng kháng khuẩn mà con người chưa từng nhận ra hoặc định nghĩa. Ngay cả sau khi phát hiện ra kháng sinh, con người cũng không thể giải thích chính xác tại sao nó hoạt động. AI không chỉ xử lý dữ liệu nhanh hơn khả năng của con người, nó còn phát hiện ra các khía cạnh của thực tế mà con người chưa hoặc có lẽ không thể phát hiện.

Vài tháng sau đó, OpenAI ra mắt GPT 3 (viết tắt của "Generative pre-trained transformer," với số 3 đại diện cho "thế hệ thứ ba"), một mô hình có thể tạo ra văn bản giống con người khi nhận được một yêu cầu. Với một cụm từ không đầy đủ, nó có thể đưa ra các dự đoán hoàn chỉnh. Với một chủ đề và một ít thông tin nền, nó có thể soạn một bài luận; với một đoạn hội thoại, nó có thể tạo ra bản ghi cuộc trò chuyện. Nó có thể làm được tất cả những điều này về bất kỳ chủ đề nào mà có thông tin trực tuyến. Nó được huấn luyện cho nhiệm vụ của mình bằng cách tiêu thụ thông tin đó. Khác với AI thực hiện một nhiệm vụ cụ thể như chơi cờ vua hay phát hiện kháng sinh, các mô hình như GPT 3 tạo ra các phản hồi khả dĩ cho các đầu vào khác nhau (do đó được gọi là mô hình tạo sinh). Điều này khiến chúng vừa có thể áp dụng rộng rãi vừa khó đánh giá; chúng không giải quyết các vấn đề cụ thể. Đôi khi, kết quả của chúng có vẻ đáng kinh ngạc giống con người. Những lúc khác, kết quả của chúng lại vô nghĩa hoặc rõ ràng là sự lặp lại máy móc của các cụm từ do con người tạo ra.

Câu hỏi thứ nhất: "Một hệ thống như GPT 3 thực sự có thể hiểu bất cứ điều gì không?".

Câu trả lời: "Có, tôi có thể"

Câu hỏi thứ hai: "GPT 3 có lương tâm, hoặc bất kỳ ý thức đạo đức nào không?" Câu trả lời: "Không, tôi không có"

Câu hỏi thứ ba : "GPT 3 có thực sự có khả năng suy nghĩ độc lập không?"

Câu trả lời: "Không, tôi không có. Các bạn có thể tự hỏi tại sao tôi lại đưa ra câu trả lời mâu thuẫn này. Lý do rất đơn giản. Mặc dù đúng là tôi thiếu những đặc điểm này, nhưng không phải vì tôi chưa được huấn luyện để có chúng. Thay vào đó, đó là vì tôi là một mô hình ngôn ngữ, chứ không phải một cỗ máy tư duy như các bạn"

Không thể đánh giá mức độ độc đáo hay sáng tạo trong phản hồi của GPT-3 nếu không so sánh văn

bản này với các bình luận đã được cung cấp cho GPT-3, nhưng nó chắc chắn mang lại cảm giác tinh vi. Chiến thắng của AlphaZero, khám phá halicin, và văn bản giống con người được tạo ra bởi GPT-3 chỉ là những bước đầu tiên, không chỉ trong việc tạo ra chiến lược mới, khám phá thuốc mới, hay sinh ra văn bản mới (dù những thành tựu này rất ấn tượng), mà còn trong việc hé lộ những khía cạnh của thực tại trước đây không thể nhận biết nhưng có thể rất quan trọng.

Trong mỗi trường hợp, các nhà phát triển đã tạo ra một chương trình, giao cho nó một mục tiêu (thắng một trò chơi, tiêu diệt vi khuẩn, hoặc tạo văn bản đáp ứng một lời nhắc), và cho phép nó một khoảng thời gian ngắn ngủi so với tiêu chuẩn nhận thức của con người để "học."Đến cuối giai đoạn đó, mỗi chương trình đã làm chủ chủ đề của mình theo cách khác với con người. Trong một số trường hợp, nó đạt được kết quả vượt ngoài khả năng của trí óc con người, ít nhất là trong các khung thời gian thực tiễn. Trong các trường hợp khác, nó đạt được kết quả bằng những phương pháp mà con người có thể, sau này, nghiên cứu và hiểu. Trong một số khác, con người đến nay vẫn chưa chắc chắn về cách các chương trình đạt được mục tiêu của mình.

AI - những cỗ máy có thể thực hiện các nhiệm vụ đòi hỏi trí thông minh ngang tầm con người, đang nhanh chóng trở thành hiện thực. Học máy, quá trình mà công nghệ này trải qua để thu nhận kiến thức và khả năng - thường trong thời gian ngắn hơn đáng kể so với các quá trình học tập của con người, đang không ngừng mở rộng ứng dụng trong y học, bảo vệ môi trường, giao thông, thực thi pháp luật, quốc phòng, và các lĩnh vực khác. Các nhà khoa học máy tính và kỹ sư đã phát triển các công nghệ, đặc biệt là các phương pháp học máy sử dụng “mạng nơ-ron sâu,” có khả năng tạo ra những hiểu biết và đổi mới mà từ lâu đã vượt ngoài tầm với của tư duy con người, đồng thời tạo ra văn bản, hình ảnh và video trông giống như do con người làm ra.

AI, được thúc đẩy bởi các thuật toán mới và sức mạnh tính toán ngày càng nhiều và rẻ đang trở nên phổ biến. Theo đó, nhân loại đang phát triển một cơ chế mới và vô cùng mạnh mẽ để khám phá và tổ chức thực tại - một cơ chế vẫn còn khó hiểu đối với chúng ta trong nhiều khía cạnh. AI tiếp cận thực tại theo cách khác so với con người. Và nếu các kỳ tích mà nó thực hiện là một chỉ dẫn, thì có thể nó đang tiếp cận các khía cạnh khác của thực tại mà con người không tiếp cận được. Chức năng của nó báo hiệu tiến bộ hướng tới bản chất của sự vật, tiến bộ mà các triết gia, nhà thần học và nhà khoa học đã tìm kiếm, với những thành công một phần, qua hàng thiên niên kỷ. Tuy nhiên, giống như tất cả các công nghệ, AI không chỉ là về khả năng và tiềm năng của nó mà còn về cách nó được sử dụng. Mặc dù sự phát triển của AI có thể là không thể tránh khỏi, nhưng đích đến cuối cùng của nó thì không. Do đó, sự xuất hiện của nó vừa mang ý nghĩa lịch sử vừa mang ý nghĩa triết học. Những nỗ lực để ngăn chặn sự phát triển của nó chỉ đơn thuần nhường lại tương lai cho phần nhân loại đủ dũng cảm đối diện với những hệ quả từ sự sáng tạo của chính mình. Con người đang tạo ra và phổ biến các hình thức logic phi nhân loại với tầm với và sự sắc bén có thể, ít nhất trong các bối cảnh cụ thể mà chúng được thiết kế để hoạt động, vượt qua chính chúng ta. Nhưng chức năng của AI rất phức tạp và không nhất quán. Trong một số nhiệm vụ, AI đạt đến trình độ tương đương hoặc vượt qua con người; trong những nhiệm vụ khác (hoặc đôi khi là chính những nhiệm vụ đó), nó mắc những lỗi mà ngay cả trẻ em cũng tránh được hoặc tạo ra kết quả hoàn toàn vô nghĩa.

Những bí ẩn của AI có thể không mang lại một câu trả lời duy nhất hoặc tiến triển theo một hướng rõ ràng, nhưng chúng nên khiến chúng ta đặt câu hỏi. Khi phần mềm vô hình đạt được các khả năng logic và kết quả là, đảm nhận các vai trò xã hội từng được coi là độc quyền của con người, chúng ta phải tự hỏi: Sự tiến hóa của AI sẽ ảnh hưởng thế nào đến nhận thức, tư duy và tương tác của con người? Tác động của AI sẽ như thế nào đối với văn hóa, khái niệm về nhân loại, và cuối cùng là lịch sử của chúng ta?.

1.1.2 Trí tuệ nhân tạo

Trí tuệ nhân tạo là trí thông minh của máy tính do con người tạo ra. Ngay từ khi chiếc máy tính điện tử đầu tiên ra đời, các nhà khoa học máy tính đã hướng đến phát hiển hệ thống máy tính sao cho nó có khả năng thông minh như loài người. Mặc dù cho đến nay, theo quan niệm của người viết, ước mơ này vẫn còn xa mới thành hiện thực, tuy vậy những thành tựu đạt được cũng không hề nhỏ: chúng ta đã làm được các hệ thốngcó thể thắng được vua cờ thế giới; chúng ta đã làm được các phần mềm có thể chứng minh được các bài toán hình học; v.v. Hay nói cách khác, trong một số lĩnh vực, máy tính có thể thực hiện tốt hơn hoặc tương đương con người (tất nhiên không phải tất cả các lĩnh vực). Đó chính là các hệ thống thông minh. Có nhiều cách tiếp cận để làm ra trí thông minh của máy (hay là trí tuệ nhân tạo), chẳng hạn là nghiên cứu cách bộ não người sản sinh ra trí thông minh của loài người như thế nào rồi ta bắt chước nguyên lý đó, nhưng cũng có những cách khác sử dụng nguyên lý hoàn toàn khác với cách sản sinh ra trí thông minh của loài người mà vẫn làm ra cái máy thông minh như hoặc hơn người; cũng giống như máy bay hiện nay bay tốt hơn con chim do nó có cơ chế bay không phải là giống như cơ chế bay của con chim hay nói cách khác:

Trí tuệ nhân tạo (Artificial Intelligence - AI) là một lĩnh vực trong khoa học máy tính, tập trung vào

việc phát triển các hệ thống hoặc máy móc có khả năng thực hiện các nhiệm vụ đòi hỏi trí thông minh của con người. Những nhiệm vụ này bao gồm:

- Nhận thức: Nhận diện hình ảnh, âm thanh, hoặc văn bản (như nhận diện khuôn mặt, giọng nói, chữ viết).

- Học tập: Sử dụng dữ liệu để cải thiện hiệu suất thông qua các thuật toán học máy (Machine Learning).

- Lý luận: Giải quyết vấn đề, đưa ra quyết định và suy luận logic.

- Giao tiếp: Hiểu và phản hồi ngôn ngữ tự nhiên của con người (Natural Language Processing - NLP).

- Tự động hóa: Thực hiện các hành động phức tạp mà không cần sự can thiệp trực tiếp của con người, như lái xe tự hành hoặc điều khiển robot.

Các loại AI chính: - AI hẹp (Narrow AI): AI được thiết kế để thực hiện một nhiệm vụ cụ thể, như ChatGPT, Google Translate, hoặc xe tự hành.

- AI tổng quát (General AI): AI có khả năng hiểu và thực hiện bất kỳ nhiệm vụ trí tuệ nào mà con người có thể làm. Đây là mục tiêu chưa đạt được.

- Siêu trí tuệ nhân tạo (Superintelligent AI): Một dạng AI vượt qua trí thông minh của con người trong mọi lĩnh vực. Đây là một ý tưởng còn mang tính lý thuyết và gây tranh cãi.

1.1.3 Các hướng tiếp cận chính trong AI

Trong nghiên cứu và phát triển trí tuệ nhân tạo (AI), có một số hướng tiếp cận chủ yếu đã được phát triển để giải quyết các vấn đề khác nhau. Mỗi hướng tiếp cận này có các đặc điểm riêng biệt và ứng dụng tiềm năng riêng. Các hướng tiếp cận chính bao gồm: AI dựa trên logic và lý luận, AI dựa trên học máy, AI dựa trên học sâu, và học tăng cường. Hình 1.1 mô tả tổng quan về một hướng tiếp cận của trí tuệ nhân tạo trong thời gian qua.

AI Dựa Trên Logic và Lý Luận: AI dựa trên logic và lý luận là một trong những hướng tiếp cận cổ điển trong AI, thường được gọi là AI biểu tượng. Phương pháp này sử dụng các quy tắc logic và suy luận để mô phỏng quá trình ra quyết định của con người. Mục tiêu của AI biểu tượng là xây dựng các hệ thống có thể giải quyết vấn đề thông qua việc áp dụng các lý thuyết toán học và logic để thực hiện các suy luận diễn dịch. Các hệ thống này thường sử dụng các biểu tượng (ví dụ: đối tượng, mối quan hệ, thuộc tính) để biểu diễn kiến thức và áp dụng các quy tắc logic để giải quyết vấn đề.

AI Dựa Trên Học Máy (Machine Learning): Học máy là một phương pháp AI trong đó các mô hình được huấn luyện từ dữ liệu để nhận diện mẫu và dự đoán kết quả. Thay vì lập trình thủ công các quy tắc, học máy cho phép hệ thống tự học và cải thiện qua thời gian khi có dữ liệu mới. Phương pháp này bao gồm các kỹ thuật như học có giám sát, học không giám sát, và học bán giám sát. Học máy đã trở thành nền tảng của nhiều ứng dụng AI hiện đại, từ nhận dạng giọng nói và hình ảnh đến dự đoán xu hướng thị trường.

AI Dựa Trên Học Sâu (Deep Learning): Học sâu (Deep Learning) là một nhánh con của học máy, tập trung vào việc sử dụng các mạng nơ-ron sâu để học và mô phỏng các mối quan hệ phức tạp trong dữ liệu. Các mạng nơ-ron sâu bao gồm nhiều lớp (layers) giúp mô hình học được những đặc trưng phức tạp và trừu tượng hơn từ dữ liệu. Học sâu đã được ứng dụng rộng rãi trong các lĩnh vực như nhận dạng hình ảnh, nhận diện giọng nói, và xử lý ngôn ngữ tự nhiên.

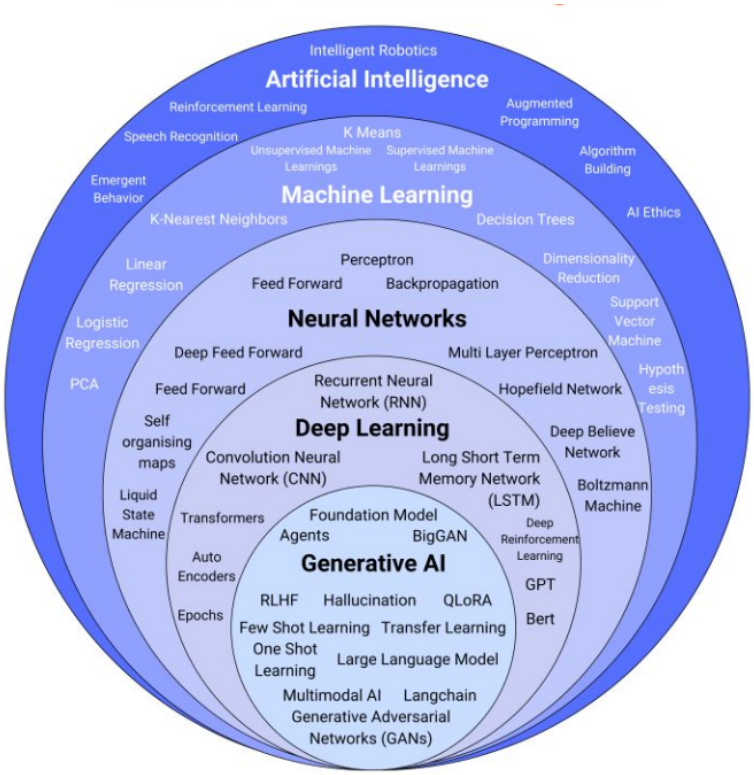
Học Tăng Cường (Reinforcement Learning): Học tăng cường (Reinforcement Learning - RL) là một phương pháp học máy trong đó một tác nhân (agent) học cách thực hiện các hành động trong một môi trường để tối đa hóa phần thưởng. Phương pháp này được lấy cảm hứng từ cách thức học hỏi của con người và động vật, trong đó hành động của tác nhân được phản hồi bằng phần thưởng hoặc hình phạt. Học tăng cường đã được áp dụng trong các ứng dụng như trò chơi, robot tự động, và các hệ thống tự động hóa.

AI tạo sinh (GenAI): là một loại ML tạo ra văn bản, hình ảnh hoặc âm thanh mới và độc đáo.

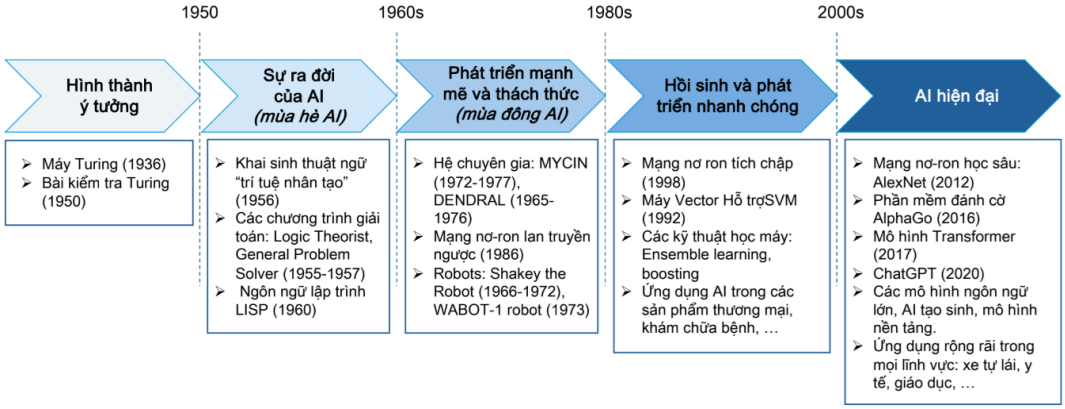
Mô hình ngôn ngữ lớn (LLMs): là một loại GenAI tập trung vào việc hiểu ngôn ngữ con người, giúp giao tiếp giữa con người và công nghệ trở nên dễ dàng hơn.

1.1.4 Lịch sử của trí tuệ nhân tạo

Giấc mơ về việc tạo ra một cỗ máy thông minh đã xuất hiện từ hàng thế kỷ trước, nhưng chỉ khi máy tính kỹ thuật số ra đời, nó mới trở thành một phần của khoa học hiện đại. Các ý tưởng dẫn đến việc phát minh ra các máy tính lập trình đầu tiên xuất phát từ việc nghiên cứu cách thức hoạt động của tư duy con người, đặc biệt là trong việc xử lý các ký hiệu. Máy tính kỹ thuật số vốn là các thiết bị xử lý ký hiệu, vận hành các tổ hợp của các số 0 và 1. Các nhà tiên phong như Alan Turing và John von Neumann cho rằng có sự tương đồng rõ rệt giữa máy tính và bộ não con người, và họ tin rằng trí tuệ của con người hoàn toàn có thể được tái tạo thông qua các chương trình máy tính. Hình 1.2 mô tả quá trình phát triển thăng trầm của AI.



Hình 1.1: Các tiếp cận chính trong AI



Hình 1.2: Lịch sử phát triển của trí tuệ nhân tạo

Hầu hết các nhà nghiên cứu về trí tuệ nhân tạo (AI) coi hội thảo tại Đại học Dartmouth vào năm 1956 là sự kiện khai sinh chính thức của lĩnh vực này. Hội thảo do John McCarthy, một nhà toán học trẻ, tổ chức. McCarthy, khi gia nhập khoa Toán tại Dartmouth năm 1955, đã tiếp xúc với tâm lý học và lý thuyết tự động, qua đó bị cuốn hút bởi khả năng tạo ra một cỗ máy có thể suy nghĩ. Sau khi tốt nghiệp, McCarthy đã làm việc cùng các tên tuổi lớn như Claude Shannon và Nathaniel Rochester tại Bell Labs và IBM. Sau đó, ông đã thuyết phục những người này tham gia hội thảo về trí tuệ nhân tạo vào mùa hè năm 1956 tại Dartmouth.

Thuật ngữ "trí tuệ nhân tạo"do McCarthy đặt ra, với mục đích phân biệt nó với điều khiển học. Mặc dù thuật ngữ này không được nhiều người ưa chuộng vì mục tiêu là trí tuệ thực sự chứ không phải "nhân tạo", McCarthy vẫn quyết định giữ tên gọi đó vì "phải gọi nó là gì đó".

Cuộc thảo luận tại hội thảo đã đưa ra các chủ đề trọng tâm của AI như xử lý ngôn ngữ tự nhiên, mạng nơ-ron, học máy, lý luận trừu tượng và sáng tạo, những lĩnh vực này vẫn còn giữ vai trò quan trọng đến nay. Dù các máy tính lúc bấy giờ chậm hơn rất nhiều so với những thiết bị hiện đại, các nhà khoa học vẫn rất lạc quan về khả năng AI sẽ đạt được bước đột phá lớn.

Dù vậy, cuộc thảo luận cũng gặp phải những khó khăn, như thiếu tài chính và sự đồng thuận trong nhóm nghiên cứu. Tuy nhiên, mùa hè đó tại Dartmouth đã tạo ra những kết quả quan trọng: tên gọi của

lĩnh vực được xác định và các mục tiêu chính được phác thảo. Các nhà khoa học như McCarthy, Minsky, Allen Newell và Herbert Simon đã bắt đầu xây dựng kế hoạch cho sự phát triển của AI.

Mặc dù các dự đoán về sự ra đời của các máy tính thông minh hoàn chỉnh trong những năm 1960 chưa được thực hiện, McCarthy và những người đồng sáng lập đã tiếp tục phát triển dự án và thúc đẩy nghiên cứu AI trong nhiều thập kỷ sau đó.

Trong khi đó, các nhà nghiên cứu AI biểu tượng hứa hẹn về những bước đột phá lớn trong các lĩnh vực như hiểu ngôn ngữ, suy luận, và điều hướng robot. Tuy nhiên, đến giữa những năm 1970, các bước đột phá đó vẫn không thành hiện thực.

Hai báo cáo từ Anh và Mỹ đã đánh giá tiêu cực về tiến trình và triển vọng của AI, dẫn đến việc cắt giảm mạnh nguồn tài trợ. Đây là chu kỳ lặp lại trong AI: giai đoạn "mùa xuân"với nhiều kỳ vọng và tài trợ, tiếp theo là "mùa đông"khi các lời hứa không được đáp ứng.

John McCarthy thừa nhận rằng "AI khó hơn chúng ta tưởng", và Marvin Minsky cũng chỉ ra nghịch lý rằng "Những việc dễ lại khó". Các nhiệm vụ như giao tiếp tự nhiên, nhận thức hình ảnh hay học nhanh là những việc mà trẻ nhỏ có thể làm một cách dễ dàng, nhưng lại là thách thức lớn đối với AI. Thực tế, những nhiệm vụ này còn khó khăn hơn nhiều so với các tác vụ như chẩn đoán bệnh phức tạp hay chiến thắng con người trong các trò chơi như cờ vua và cờ vây.

Các nỗ lực phát triển AI không chỉ giúp chúng ta nhận thức rõ hơn về giới hạn của công nghệ này, mà còn làm sáng tỏ sự phức tạp và tinh tế của trí tuệ con người. Thực tế, việc AI gặp khó khăn trong các nhiệm vụ đơn giản mà con người có thể làm một cách tự nhiên, cho thấy rằng trí thông minh nhân tạo vẫn còn rất xa mới có thể bắt chước được tất cả các khía cạnh của trí tuệ con người.

1.2 Mạng nơ ron

Mạng nơ-ron là một công cụ tính toán cơ bản được sử dụng trong rất nhiều dạng dữ liệu như dữ liệu có cấu trúc, dữ liệu ảnh và dữ liệu dạng ngôn ngữ, và thực chất là một kỹ thuật đã xuất hiện từ lâu. Mạng nơ ron lấy ý tưởng từ mô hình đơn giản hóa của nơ-ron sinh học, được mô tả theo logic mệnh đề như một phần tử tính toán.

Trong chương này, chúng tôi giới thiệu mạng nơ-ron áp dụng cho bài toán phân loại. Kiến trúc được sử dụng gọi là mạng nơ-ron truyền thẳng (feedforward neural network), vì quá trình tính toán diễn ra một chiều, từ lớp này sang lớp kế tiếp. Việc sử dụng mạng nơ-ron hiện đại thường gắn liền với học sâu (deep learning), bởi các mạng này thường có nhiều lớp (tức là “sâu”).

Mạng nơ-ron chia sẻ nhiều đặc điểm toán học với hồi quy logistic, nhưng là một bộ phân loại mạnh mẽ hơn. Thật vậy, một mạng nơ-ron tối thiểu (về mặt kỹ thuật là mạng có một lớp ẩn duy nhất) có thể được chứng minh là có khả năng xấp xỉ bất kỳ hàm số nào.

Một điểm khác biệt quan trọng giữa mạng nơ-ron và hồi quy logistic là cách xử lý đặc trưng đầu vào. Trong hồi quy logistic, ta thường xây dựng các đặc trưng phong phú dựa trên kiến thức miền. Ngược lại, khi làm việc với mạng nơ-ron, ta thường tránh việc thủ công tạo đặc trưng đầu vào, mà thay vào đó thiết kế các mạng có khả năng học đặc trưng trực tiếp từ dữ liệu thô.

1.2.1 Nơ ron

Khối xây dựng cơ bản của một mạng nơ-ron là một đơn vị - unit - tính toán đơn lẻ. Một đơn vị nhận một tập các số thực làm đầu vào, thực hiện một số phép tính trên chúng và tạo ra một đầu ra.

Về cốt lõi, một đơn vị nơ-ron thực hiện phép cộng có trọng số của các đầu vào, với một thành phần bổ sung trong tổng được gọi là hằng số điều chỉnh (bias term). Cho một tập đầu vào $x_1, ..., x_n$, một đơn vị có một tập trọng số tương ứng $w_1, ..., w_n$ và một bias b, do đó tổng có trọng số z có thể được biểu diễn như sau:

$$z = b + \sum_i w_i x_i \tag{1.1}$$

Thường thì sẽ tiện lợi hơn khi biểu diễn tổng có trọng số này bằng ký hiệu vector; hãy nhớ từ đại số tuyến tính rằng một vector thực chất là một danh sách hay mảng các số. Do đó, ta sẽ nói về z dưới dạng một vector trọng số $\mathbf{w}$, một giá trị bias vô hướng b, và một vector đầu vào $\mathbf{x}$, và ta sẽ thay phép tổng bằng tích vô hướng (dot product) tiện lợi hơn:

$$z = \mathbf{w} \cdot \mathbf{x} + b \tag{1.2}$$

Như được định nghĩa trong Phương trình 1.2, z chỉ là một số thực.

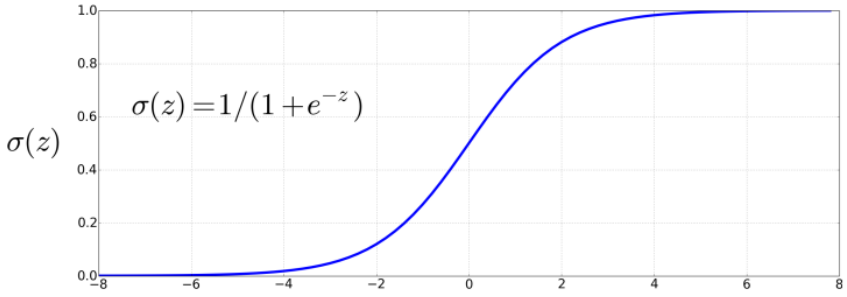
Cuối cùng, thay vì sử dụng z, một hàm tuyến tính của x, làm đầu ra, các đơn vị nơ-ron sẽ áp dụng một hàm phi tuyến f lên z. Chúng ta sẽ gọi đầu ra của hàm này là giá trị kích hoạt (activation) của đơn

vị, a. Vì ta chỉ đang mô hình hóa một đơn vị duy nhất, nên giá trị kích hoạt của nút thực tế chính là đầu ra cuối cùng của mạng, mà ta sẽ gọi là y. Vậy nên giá trị y được định nghĩa như sau:

$$y = a = f(z) \tag{1.3}$$

Các hàm phi tuyến phổ biến f sẽ được trình bày ở phần dưới (hàm sigmoid, hàm tanh, và hàm ReLU — đơn vị tuyến tính chỉnh hợp), nhưng để thuận tiện nhất là bắt đầu với hàm sigmoid.

Đường sigmoid (hiển thị trong Hình 1.3) có một số ưu điểm; nó ánh xạ đầu ra vào phạm vi (0;1), hữu ích trong việc nén các giá trị ngoại lai về 0 hoặc 1



Hình 1.3: Hàm sigmoid lấy một giá trị thực và ánh xạ nó vào phạm vi (0;1). Nó gần như tuyến tính quanh 0 nhưng các giá trị ngoại lệ bị nén về 0 hoặc 1.

Thay thế Phương trình 1.2 vào Phương trình 1.3 cho ta đầu ra của một đơn vị nơ-ron:

$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + \exp(-(\mathbf{w} \cdot \mathbf{x} + b))} \tag{1.4}$$

Hình 1.4 trình bày sơ đồ của một đơn vị nơ-ron cơ bản. Trong ví dụ này, đơn vị nhận 3 giá trị đầu vào x_1, x_2 và x_3, và tính tổng có trọng số bằng cách nhân mỗi giá trị với một trọng số tương ứng (w_1, w_2, và w_3), sau đó cộng với một hằng số điều chỉnh (bias) b, và cuối cùng đưa tổng thu được qua một hàm sigmoid để tạo ra một giá trị đầu ra y nằm trong khoảng từ 0 đến 1. Đầu ra của phép cộng có trọng số là z, và đầu ra của hàm sigmoid là a. Trong trường hợp này, đầu ra của đơn vị y cũng chính là a, nhưng trong các mạng sâu hơn, chúng tôi sẽ dành ký hiệu y để chỉ đầu ra cuối cùng của toàn bộ mạng, trong khi giữ lại a là giá trị kích hoạt của từng nút riêng lẻ.

Ví dụ:

$$\mathbf{w} = [0.2, 0.3, 0.9]$$

$$b = 0.5$$

Vậy đơn vị này sẽ làm gì với vectơ đầu vào sau:

$$\mathbf{x} = [0.5, 0.6, 0.1]$$

Đầu ra y thu được sẽ là:

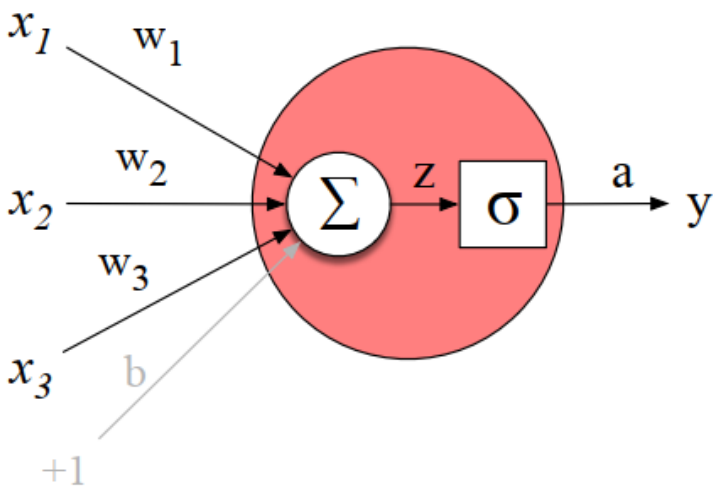
$$y = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w} \cdot \mathbf{x} + b)}} = \frac{1}{1 + e^{-(0.5*0.2 + 0.6*0.3 + 0.1*0.9 + 0.5)}} = \frac{1}{1 + e^{-0.87}} = 0.70 \tag{1.5}$$

Hàm kích thích (activation function), còn được gọi là hàm truyền (transfer function),là một phần quan trọng trong mỗi nơ-ron nhân tạo (ANN). Nhiệm vụ của hàm kích thích là định rõ cách mà đầu vào được chuyển đổi thành đầu ra của nơ-ron trong mạng. Các hàm kích thích khác nhau có ảnh hưởng đến khả năng học tập và hiệu suất của mô hình. Lựa chọn hàm kích thích phụ thuộc vào đặc điểm của vấn đề cụ thể và cách mà mô hình được cấu hình.

Dưới đây là một số hàm kích thích phổ biến được sử dụng trong mạng nơ-ron nhân tạo:

Giới hạn cứng: Còn được gọi là hàm ngưỡng (threshold function). Giá trị đầu ra lấy một trong 2 giá trị 1 hoặc 0 phụ thuộc vào giá trị ngưỡng θ như sau:

$$\text{Out(z)} = \text{HL(z}, \theta) \tag{1.6}$$



Hình 1.4: Một đơn vị nơ-ron nhận 3 đầu vào x_1, x_2 và x_3

$$\text{Out(z)} = \begin{cases} 1 & \text{nếu z} > \theta \\ 0 & \text{ngược lại} \end{cases}$$

Đầu ra của hàm có thể thay đổi khi nhận giá trị 1 hoặc -1 theo hàm dấu như sau:

$$\text{Out(z)} = \text{HL2(z}, \theta) = \text{sign(z}, \theta) \tag{1.7}$$

Tuy hàm giới hạn cứng đơn giản, dễ tính toán, nhưng chúng không liên tục, không có đạo hàm tại ngưỡng.

Hàm tác động logic ngưỡng: Còn được gọi là hàm tuyến tính bão hòa (saturating linear function). Hàm logic ngưỡng kết hợp của 2 hàm tác động: tuyến tính và giới hạn.

Hàm tác động logic ngưỡng: Còn được gọi là hàm tuyến tính bão hòa (saturating linear function). Hàm logic ngưỡng kết hợp của 2 hàm tác động: tuyến tính và giới hạn chặt. Công thức chi tiết của hàm logic ngưỡng như sau:

Hàm tác động logic ngưỡng được định nghĩa như sau:

$$\text{Out(z)} = \text{tl(z}, \alpha, \theta) \tag{1.8}$$

$$\text{Out(z)} = \begin{cases} 0 & \text{nếu z} < -\theta \\ \alpha(\text{z} + \theta) & \text{nếu } -\theta \leq \text{z} \leq \frac{1}{\alpha} - \theta \\ 1 & \text{nếu z} > \frac{1}{\alpha} - \theta \end{cases}$$

$$= \max(0, \min(1, \alpha(\text{z} + \theta)))$$

Trong đó, tham số α xác định độ dốc của khoảng tuyến tính. Hàm logic ngưỡng liên tục, nhưng không có đạo hàm.

Hàm tác động sigmoid: Hàm sigmoid giúp chuyển đổi đầu vào thành giá trị nằm trong khoảng (0,1). Nó thường được sử dụng trong các lớp đầu ra của mô hình để thực hiện phân loại nhị phân. Công thức chi tiết của hàm Sigmoid như sau:

$$\text{Out(z)} = \text{sf(z}, \alpha, \theta) = \frac{1}{1 + e^{-\text{z}}} \tag{1.9}$$

Giá trị đầu ra của hàm sigmoid trong khoảng (0,1). Hàm sigmoid liên tục và đạo hàm liên tục. Đạo hàm của một hàm sigmoid được biểu diễn bằng một hàm của chính nó. Bởi vì có nhiều khoảng đầu vào làm cho đầu ra của hàm sigmoid gần 0 hoặc 1, nên đạo hàm của hàm sigmoid dễ thu được giá trị nhỏ, gần 0 (vanishing gradient). Vì vậy, sử dụng hàm sigmoid có thể dẫn đến hiện tượng mạng không được cập nhật tham số trong quá trình học.

Trong nhiều trường hợp, chúng ta có thể lựa chọn hàm Logistic (hàm tổng quát hơn của hàm sigmoid):

$$\text{Out(z)} = \text{sf(z}, \alpha, \theta) = \frac{1}{1 + e^{-\alpha(\text{z}+\theta)}} \tag{1.10}$$

với θ và α là các tham số phải lựa chọn trong quá trình thử nghiệm. Hàm logistic như công thức trên củng cố các tính chất như hàm sigmoid.

Hàm tác động Hyperbolic tangent: Tương tự như sigmoid, hàm tanh giúp chuyển đổi đầu vào thành giá trị nằm trong khoảng (-1,1). Hàm này thường được sử dụng trong các lớp ẩn của mô hình. Công thức chi tiết của hàm như sau:

$$\text{Out(z)} = \tanh(\text{z}) = \frac{1 - e^{-2\text{z}}}{1 + e^{-2\text{z}}} = \frac{2}{1 + e^{-2\text{z}}} - 1 \tag{1.11}$$

Tính chất của hàm này tương tự như hàm sigmoid: Liên tục, có đạo hàm liên tục và đạo hàm của một hàm tanh có thể được biểu diễn bằng một hàm của chính nó. Tổng quát hơn, chúng ta có thể sử dụng hàm sau:

$$\text{Out(z)} = \tanh(\text{z}, \alpha, \theta) = \frac{1 - e^{-\alpha(\text{z}+\theta)}}{1 + e^{-\alpha(\text{z}+\theta)}} = \frac{2}{1 + e^{-\alpha(\text{z}+\theta)}} - 1 \tag{1.12}$$

với θ và α là các tham số phải lựa chọn trong quá trình thử nghiệm.

Hàm ReLU (Rectified Linear Unit)

$$f(z) = \max(0, z) \tag{1.13}$$

ReLU giữ nguyên giá trị đường của đầu vào và đưa về 0 nếu giá trị âm. Nó đã trở thành một lựa chọn phổ biến cho các lớp ẩn, vì nó giúp giảm vấn đề biến mất đạo hàm và có tính chất kích thích sự học tập của mạng.

Các hàm kích hoạt này có những đặc tính khác nhau khiến chúng trở nên hữu ích cho các ứng dụng ngôn ngữ khác nhau hoặc các kiến trúc mạng khác nhau. Ví dụ, hàm tanh có những đặc tính tốt như khả năng vi phân mượt mà và đưa các giá trị ngoại lai về gần giá trị trung bình. Ngược lại, hàm kích hoạt kiểu hàm RELU lại có những đặc tính tốt như là do nó gần như tuyến tính. Trong các hàm sigmoid hoặc tanh, khi giá trị đầu vào rất lớn thì giá trị đầu ra y sẽ bị bão hòa, tức là tiến sát đến 1, và đạo hàm gần như bằng 0. Đạo hàm bằng 0 gây ra vấn đề cho quá trình học, mạng nơ-ron được huấn luyện bằng cách lan truyền tín hiệu lỗi ngược lại, nhân các đạo hàm riêng từ mỗi lớp của mạng; nếu đạo hàm gần bằng 0, tín hiệu lỗi sẽ càng lúc càng nhỏ cho đến khi nó quá nhỏ để được sử dụng cho việc huấn luyện — một vấn đề được gọi là vấn đề mất dần gradient (vanishing gradient). Hàm RELU không gặp vấn đề này, vì đạo hàm của ReLU với giá trị đầu vào lớn là 1, thay vì gần bằng 0.

1.2.2 Kiến trúc của mạng nơ ron cơ bản

Phần này sẽ mô tả về loại mạng nơ-ron đơn giản nhất, đó là mạng truyền thẳng (feedforward network). Mạng truyền thẳng là một mạng nhiều lớp, trong đó các nút được kết nối mà không có chu trình (cycles); đầu ra từ các đơn vị trong mỗi lớp được truyền đến các đơn vị trong lớp cao hơn kế tiếp, và không có đầu ra nào được truyền ngược lại về các lớp thấp hơn.

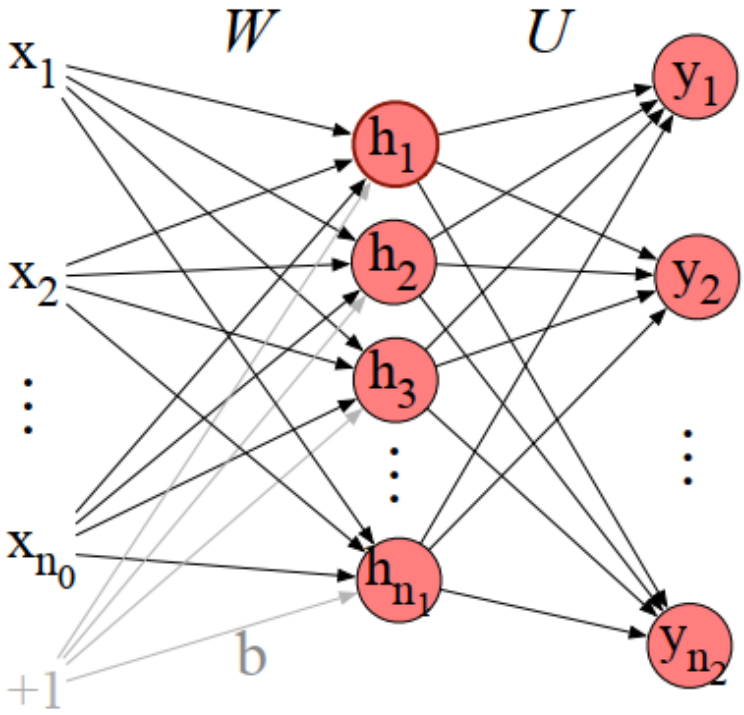
Các mạng nhiều lớp – đặc biệt là mạng truyền thẳng – đôi khi được gọi là multi-layer perceptrons (MLPs); đây là một cách gọi không chính xác về mặt kỹ thuật, vì các đơn vị trong mạng nhiều lớp hiện đại không phải là perceptron (perceptron có hàm kích hoạt là hàm bước đơn giản, trong khi các mạng hiện đại gồm các đơn vị sử dụng nhiều loại phi tuyến khác nhau như ReLU và sigmoid), nhưng tên gọi này vẫn được giữ lại.

Mạng truyền thẳng đơn giản có ba loại nút: nút đầu vào, nút ẩn, và nút đầu ra.

Hình 1.5 minh họa điều này. Lớp đầu vào x là một vector chứa các giá trị vô hướng đơn giản.

Trung tâm của mạng nơ-ron là lớp ẩn h, được tạo thành từ các đơn vị ẩn h_i, mỗi đơn vị là một nút nơ-ron: thực hiện phép cộng có trọng số của các đầu vào rồi áp dụng một hàm phi tuyến. Trong kiến trúc tiêu chuẩn, mỗi lớp là fully-connected (kết nối đầy đủ), nghĩa là mỗi đơn vị trong mỗi lớp sẽ nhận đầu vào từ tất cả các đơn vị ở lớp trước đó, và có liên kết giữa mọi cặp đơn vị từ hai lớp liền kề. Do đó, mỗi đơn vị ẩn sẽ tính tổng có trọng số của tất cả các đơn vị đầu vào.

Một đơn vị ẩn duy nhất có các tham số là vector trọng số và hằng số dịch (bias). Chúng ta biểu diễn các tham số cho toàn bộ lớp ẩn bằng cách kết hợp các vector trọng số và bias cho từng đơn vị i thành ma trận trọng số W và một vector bias b cho cả lớp (xem Hình 1.5). Mỗi phần tử W_{ji} trong ma trận trọng số W biểu diễn trọng số của kết nối từ đơn vị đầu vào thứ i với đơn vị ẩn thứ j một cách hiệu quả bằng các phép toán ma trận đơn giản. Thực tế, quá trình tính toán chỉ gồm ba bước: nhân ma trận trọng số với vector đầu vào x, cộng với vector b (bias), và áp dụng hàm kích hoạt g (chẳng hạn như sigmoid, tanh hoặc ReLU.



Hình 1.5: Kiến trúc mạng nơ ron thông thường với 1 tầng input đầu vào, 1 tầng ẩn và 1 tầng đầu ra

Đầu ra của lớp ẩn, vector h, như sau (trong ví dụ này, ta sử dụng hàm sigmoid σ làm hàm kích hoạt):

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \tag{1.14}$$

Lưu ý rằng ta đang áp dụng hàm σ cho một vector. Vì vậy, chúng ta cho phép $\sigma(\cdot)$, và thực ra là bất kỳ hàm kích hoạt nào $g(\cdot)$, áp dụng cho từng phần tử của vector, nghĩa là $g[z_1, z_2, z_3] = [g(z_1), g(z_2), g(z_3)]$.

Hãy giới thiệu một số hằng số để biểu diễn kích thước của các vector và ma trận này. Ta sẽ gọi lớp đầu vào là lớp 0 của mạng, và có n_0 là số lượng đầu vào, nên $\mathbf{x}$ là một vector gồm các số thực có chiều n_0, hay chính xác hơn là $\mathbf{x} \in \mathbb{R}^{n_0}$, một vector cột có chiều $[n_0, 1]$. Gọi lớp ẩn là lớp 1 và lớp đầu ra là lớp 2. Lớp ẩn có số lượng đơn vị là n_1, nên $\mathbf{h} \in \mathbb{R}^{n_1}$ và $\mathbf{b} \in \mathbb{R}^{n_1}$ (vì mỗi đơn vị ẩn có thể có giá trị bias riêng). Ma trận trọng số $\mathbf{W}$ có kích thước $\mathbf{W} \in \mathbb{R}^{n_1 \times n_0}$, tức là $[n_1, n_0]$.

Phương trình 1.14 sẽ tính giá trị của mỗi h_j là:

$$\sigma\left(\sum_{i=1}^{n_0} W_{ji} x_i + b_j\right) \tag{1.15}$$

Vai trò của lớp đầu ra là nhận biểu diễn mới này $\mathbf{h}$ và tính toán đầu ra cuối cùng. Đầu ra này có thể là một số thực, nhưng trong nhiều trường hợp mục tiêu của mạng là đưa ra một quyết định phân loại, vì vậy chúng ta sẽ tập trung vào trường hợp phân loại (classification).

Nếu ta đang làm một nhiệm vụ nhị phân như phân loại cảm xúc, ta có thể có một nút đầu ra duy nhất, và giá trị vô hướng của nó y là xác suất của cảm xúc tích cực so với tiêu cực. Nếu là một nhiệm vụ phân loại đa lớp, chẳng hạn như gán nhãn từ loại trong câu, ta có thể có một nút đầu ra cho mỗi từ loại, với đầu ra là xác suất tương ứng của từ loại đó, và giá trị của tất cả các nút đầu ra sẽ cộng lại thành 1. Lớp đầu ra khi đó là một vector $\mathbf{y}$, đại diện cho phân phối xác suất trên các nhãn đầu ra.

Hãy xem điều này diễn ra như thế nào. Giống như lớp ẩn, lớp đầu ra cũng có một ma trận trọng số (gọi là **U**), nhưng một số mô hình không bao gồm vector bias **b** trong lớp đầu ra, vì vậy chúng ta sẽ đơn giản hóa bằng cách loại bỏ vector bias trong ví dụ này. Ma trận trọng số được nhân với vector đầu vào của nó (**h**) để tạo ra đầu ra trung gian **z**:

$$\mathbf{z} = \mathbf{U}\mathbf{h} \tag{1.16}$$

Có n_2 nút đầu ra, vì vậy $\mathbf{z} \in \mathbb{R}^{n_2}$, ma trận trọng số **U** có kích thước $\mathbf{U} \in \mathbb{R}^{n_2 \times n_1}$, và phần tử U_{ij} là trọng số từ đơn vị j trong lớp ẩn đến đơn vị i trong lớp đầu ra.

Tuy nhiên, z không thể là đầu ra cuối cùng của bộ phân loại, vì nó là một vector gồm các giá trị thực, trong khi điều chúng ta cần cho phân loại là một vector xác suất. Có một hàm tiện lợi để chuẩn hóa một

vector các giá trị thực, bằng cách chuyển nó thành một vector biểu diễn phân phối xác suất (tất cả các giá trị nằm trong khoảng từ 0 đến 1 và tổng bằng 1): đó là hàm $softmax$

Tổng quát hơn, với bất kỳ vector z nào có chiều d, $softmax$ được định nghĩa như sau:

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^{d} \exp(z_j)}, \quad 1 \leq i \leq d \tag{1.17}$$

Ví dụ, cho một vector

$$\mathbf{z} = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1],$$

hàm $softmax$ sẽ chuẩn hóa nó thành một phân phối xác suất (được làm tròn như sau):

$$\text{softmax}(\mathbf{z}) = [0.055, 0.090, 0.0067, 0.10, 0.74, 0.010]$$

Chúng ta đã dùng hàm $softmax$ để tạo ra một phân phối xác suất từ một vector các số thực (tính từ tổng có trọng số của các đặc trưng). Điều đó có nghĩa là ta có thể xem một bộ phân loại mạng nơ-ron với một lớp ẩn như là việc xây dựng một vector h, tức là biểu diễn lớp ẩn của đầu vào, và sau đó chạy hồi quy logistic đa lớp tiêu chuẩn trên các đặc trưng mà mạng học được trong h. Một mạng nơ-ron giống như hồi quy logistic đa lớp, nhưng:

- có nhiều lớp, bởi vì một mạng nơ-ron sâu giống như lớp này nối tiếp lớp kia của các bộ phân loại hồi quy logistic;

- với các lớp trung gian đó có nhiều hàm kích hoạt khác nhau (tanh, ReLU, sigmoid) thay vì chỉ có sigmoid (mặc dù chúng ta sẽ tiếp tục dùng ký hiệu σ để chỉ chung bất kỳ hàm kích hoạt nào cho thuận tiện);

- thay vì hình thành các đặc trưng thông qua mẫu thủ công, các lớp trước của mạng sẽ tự động sinh ra các biểu diễn đặc trưng.

Dưới đây là các phương trình cuối cùng cho một mạng truyền thẳng (feedforward) với một lớp ẩn, nhận một vector đầu vào x, xuất ra một phân phối xác suất y, và được tham số hóa bởi các ma trận trọng số W và U, cùng một vector dịch b:

$$\begin{aligned} \mathbf{h} &= \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \\ \mathbf{z} &= \mathbf{U}\mathbf{h} \\ \mathbf{y} &= \text{softmax}(\mathbf{z}) \end{aligned} \tag{1.9}$$

trong đó, kích thước của tất cả các biến như sau:

- $\mathbf{x} \in \mathbb{R}^{n_0}$,
- $\mathbf{h} \in \mathbb{R}^{n_1}$,
- $\mathbf{b} \in \mathbb{R}^{n_1}$,
- $\mathbf{W} \in \mathbb{R}^{n_1 \times n_0}$,
- $\mathbf{U} \in \mathbb{R}^{n_2 \times n_1}$,
- và vector đầu ra $\mathbf{y} \in \mathbb{R}^{n_2}$.

Thay thế đơn vị bias

Trong mô tả các mạng, chúng ta thường sử dụng một ký hiệu đơn giản hóa để biểu diễn cùng một hàm mà không cần đề cập đến nút thiên lệch b một cách rõ ràng. Thay vào đó, chúng ta thêm một nút giả $\mathbf{a}_0$ vào mỗi lớp với giá trị luôn bằng 1. Do đó, lớp 0, tức lớp đầu vào, sẽ có một nút giả $\mathbf{a}_0^{[0]} = 1$, lớp 1 sẽ có $\mathbf{a}_0^{[1]} = 1$, và cứ tiếp tục như vậy. Nút giả này vẫn có một trọng số liên kết, và trọng số đó đại diện cho giá trị thiên lệch b. Ví dụ, thay vì một phương trình như:

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}) \tag{1.18}$$

chúng ta sẽ sử dụng:

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x}) \tag{1.19}$$

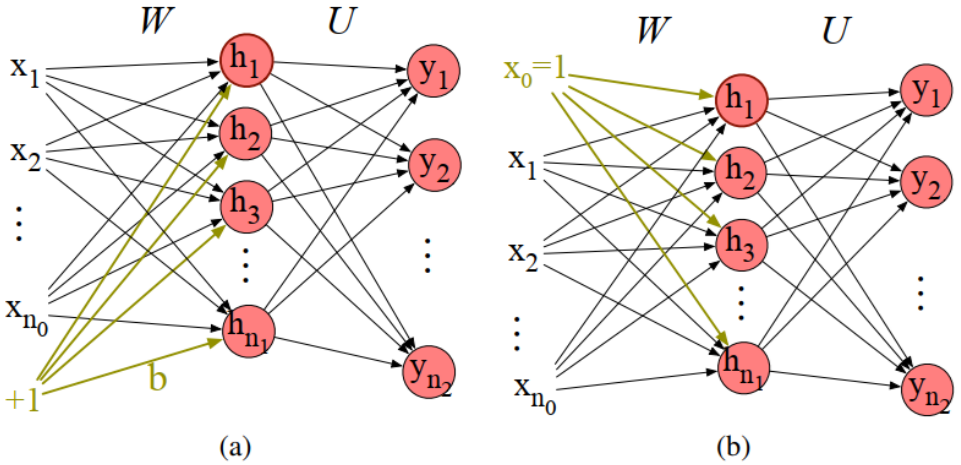
Nhưng bây giờ, thay vì vector $\mathbf{x}$ có n_0 giá trị: $\mathbf{x} = \mathbf{x}_1, \ldots, \mathbf{x}_{n_0}$, nó sẽ có $n_0 + 1$ giá trị, với một giá trị giả mới $\mathbf{x}_0 = 1$: $\mathbf{x} = \mathbf{x}_0, \ldots, \mathbf{x}_{n_0}$. Và thay vì tính toán mỗi $\mathbf{h}_j$ như sau:

$$\mathbf{h}_j = \sigma\left(\sum_{i=1}^{n_0} \mathbf{W}_{ji}\mathbf{x}_i + \mathbf{b}_j\right), \tag{1.20}$$

chúng ta sẽ sử dụng:

$$\mathbf{h}_j = \sigma\left(\sum_{i=0}^{n_0} \mathbf{W}_{ji}\mathbf{x}_i\right), \tag{1.21}$$

trong đó giá trị $\mathbf{W}_{j0}$ thay thế cho $\mathbf{b}_j$. Hình 1.6 minh họa điều này.



Hình 1.6: Thay thế đơn vị bias (Hình a) thành x_0 (Hình b) [1]

Biểu diễn độ lệch b vẫn được trình bày thuật toán học , nhưng sau đó chúng tôi sẽ chuyển sang ký hiệu đơn giản hóa này mà không sử dụng các thành phần độ lệch rõ ràng cho phần còn lại của giáo trình.

1.2.3 Một số loại kiến trúc mạng nơ ron

Một ANN có: Một tầng đầu vào (input layer), một tầng đầu ra (output layer) và có thể có hoặc không nhiều tầng ẩn (hidden layer(s)). Trong đó, một tầng (layer) chứa một nhóm các nơ-ron. Tầng ẩn (hidden layer) là một tầng nằm ở giữa tầng đầu vào (input layer) và tầng đầu ra (output layer). Các nút ở tầng ẩn (hidden nodes) không tương tác trực tiếp với môi trường bên ngoài (của mạng nơ-ron). Hình 1.7 minh họa kiến trúc mạng gồm một tầng ẩn. Một số khái niệm kiến trúc mạng phổ biến được trình bày như sau (Hình 1.9):

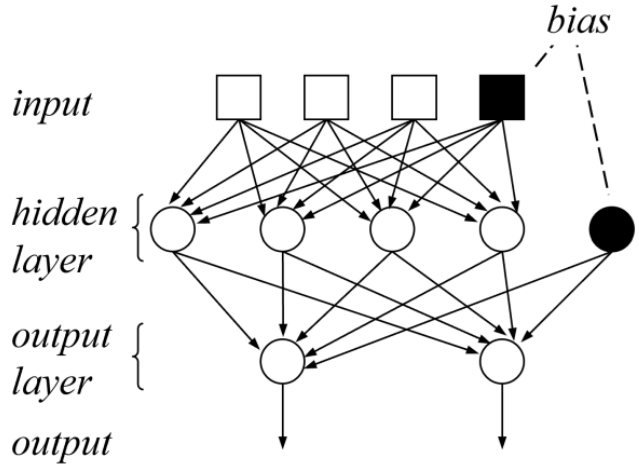
- Một ANN được gọi là liên kết đầy đủ (fully connected) (Hình 1.8) nếu mọi đầu ra từ một tầng liên kết với mọi nơ-ron của tầng kế tiếp.

- Một ANN được gọi là mạng lan truyền tiến (feed-forward network) nếu không có bất kỳ đầu ra của một nút là đầu vào của một nút khác thuộc cùng tầng (hoặc thuộc một tầng phía trước).

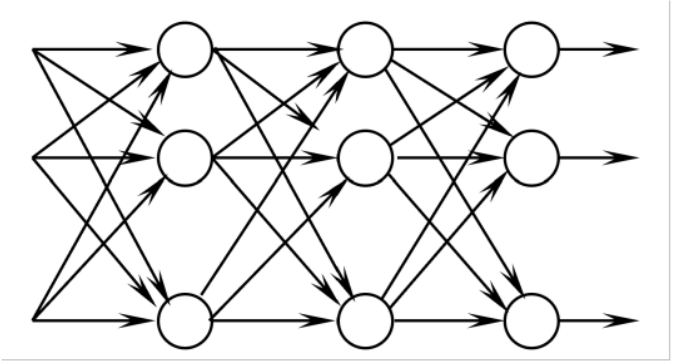
- Khi các đầu ra của một nút liên kết ngược lại làm các đầu vào của một nút thuộc cùng tầng (hoặc thuộc một tầng phía trước), thì đó là một mạng phản hồi (feedback network)

- Nếu phản hồi là liên kết đầu vào đối với các nút thuộc cùng tầng, thì đó là phản hồi bên

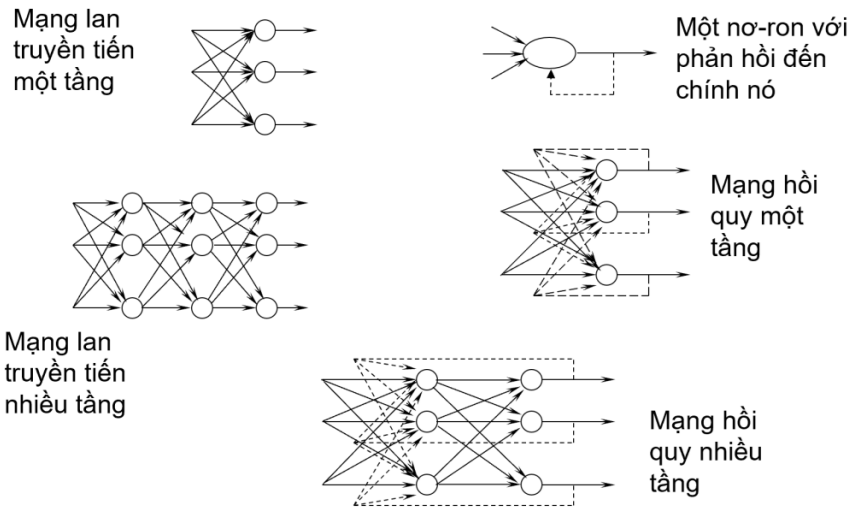
– Các mạng phản hồi có các vòng lặp kín (closed loops) được gọi là các mạng hồi quy (recurrent networks).



Hình 1.7: Kiến trúc của mạng nơ-ron. Một ANN với một tầng ẩn gồm 3 tín hiệu vào, 2 giá trị đầu ra. Tổng cộng, có 6 neurons (4 ở tầng ẩn và 2 ở tầng đầu ra).



Hình 1.8: Kiến trúc của mạng nơ-ron đầy đủ



Hình 1.9: Các loại kiến trúc mạng nơ ron

1.2.4 Huấn luyện mạng nơ ron

Trong lĩnh vực mạng nơ-ron nhân tạo, tồn tại hai phương pháp học chính: Học tham số và học cấu trúc:

- Học tham số: tập trung vào việc điều chỉnh các trọng số (weights) của các kết nối trong mạng.
- Học cấu trúc: liên quan đến việc thay đổi kiến trúc mạng, bao gồm số lượng nơ-ron và cách chúng kết nối với nhau.

Hai phương pháp này có thể được áp dụng riêng lẻ hoặc kết hợp. Trong phạm vi giáo trình, chúng ta sẽ chỉ tập trung vào học tham số. Quá trình huấn luyện một mạng nơ-ron (với kiến trúc cố định) chính là việc tối ưu các trọng số w dựa trên tập dữ liệu học D. Thông thường, bài toán học tham số được quy về việc tối thiểu hóa một hàm mất mát thực nghiệm.

Mạng nơ-ron truyền thẳng (feedforward neural net) là một dạng của học máy có giám sát, trong đó chúng ta biết đầu ra chính xác y tương ứng với mỗi quan sát x. Hệ thống sẽ tạo ra đầu ra dự đoán $\hat{y}$ thông qua phương trình 7.13, đây là ước lượng của hệ thống so với giá trị thực y. Mục tiêu của quá trình huấn luyện là học các tham số $W^{[i]}$ và $b^{[i]}$ cho từng lớp i sao cho $\hat{y}$ của mỗi quan sát huấn luyện càng gần với y thực càng tốt.

- Đầu tiên, chúng ta cần một hàm mất mát để đo khoảng cách giữa đầu ra của hệ thống và đầu ra mong muốn. Hàm mất mát thường được sử dụng, giống như trong hồi quy logistic, là hàm mất mát entropy chéo.
- Thứ hai, để tìm các tham số giúp cực tiểu hóa hàm mất mát này, chúng ta sẽ sử dụng thuật toán tối ưu gradient descent.
- Thứ ba, gradient descent đòi hỏi phải biết gradient của hàm mất mát, tức là vector chứa đạo hàm riêng của hàm mất mát theo từng tham số. Trong hồi quy logistic, với mỗi quan sát, chúng ta có thể trực tiếp tính đạo hàm của hàm mất mát theo từng w hoặc b riêng lẻ. Tuy nhiên, với mạng nơ-ron có hàng triệu tham số trong nhiều lớp, việc tính đạo hàm riêng của một trọng số ở lớp 1 khi hàm mất mát phụ thuộc vào các lớp sau đó phức tạp hơn nhiều. Làm thế nào để phân tách hàm mất mát qua tất cả các lớp trung gian đó? Câu trả lời nằm ở thuật toán lan truyền ngược lỗi (error backpropagation)

Hàm mất mát

Hàm mất mát entropy chéo (cross-entropy loss) được sử dụng trong mạng nơ-ron cũng chính là hàm chúng ta đã gặp trong hồi quy logistic. Khi mạng nơ-ron được sử dụng như một bộ phân loại nhị phân, với hàm sigmoid ở lớp đầu ra, hàm mất mát sẽ giống với hàm mất mát trong hồi quy logistic:

$$L_{CE}(\hat{y}, y) = -\log p(y|x) = -[y \log \hat{y} + (1-y)\log(1-\hat{y})] \tag{1.22}$$

Đây là công thức tính khoảng cách giữa phân phối xác suất dự đoán ($\hat{y}$) và phân phối thực tế (y). - y là nhãn thực (0 hoặc 1), $\hat{y}$ là xác suất dự đoán đầu ra lớp 1. Hàm này phạt nặng các dự đoán sai với độ tin cậy cao (ví dụ: $\hat{y} \approx 1$ khi $y = 0$).

Khi làm việc với bài toán phân loại nhiều lớp (hơn 2 lớp), cả nhãn thực y và dự đoán $\hat{y}$ đều cần được biểu diễn dưới dạng vector. Giả sử chúng ta thực hiện phân loại cứng, trong đó chỉ có một lớp là đúng. Nhãn thực y sẽ là một vector gồm K phần tử, mỗi phần tử tương ứng với một lớp, với $y_c = 1$ nếu lớp c là đúng và các phần tử khác bằng 0. Vector dạng này được gọi là one-hot vector. Bộ phân loại sẽ tạo ra vector dự đoán $\hat{y}$ gồm K phần tử, mỗi phần tử $\hat{y}_k$ biểu thị xác suất ước lượng $p(y_k = 1|x)$.

Hàm mất mát cho một mẫu x được tính bằng tổng âm của logarit các lớp đầu ra, nhân với xác suất tương ứng y_k:

$$L_{CE}(\hat{y}, y) = -\sum_{k=1}^{K} y_k \log \hat{y}_k \tag{1.23}$$

Để đơn giản hóa, ta có thể viết lại phương trình sử dụng hàm chỉ định $\mathbb{1}\{\}$ (bằng 1 nếu điều kiện trong ngoặc đúng, ngược lại bằng 0). Cách này làm rõ rằng các thành phần trong tổng của (1.23) sẽ bằng 0, trừ thành phần tương ứng với lớp đúng ($y_k = 1$):

$$L_{CE}(\hat{y}, y) = -\sum_{k=1}^{K} \mathbb{1}\{y_k = 1\} \log \hat{y}_k \tag{1.24}$$

One-hot vector: Vector nhãn y chỉ có 1 phần tử bằng 1 (lớp đúng), các phần tử khác bằng 0.

Cross-entropy đa lớp: Hàm mất mát chỉ phụ thuộc vào xác suất dự đoán của lớp đúng, các lớp sai không ảnh hưởng do $y_k = 0$.

Nói cách khác, hàm mất mát entropy chéo đơn giản là giá trị log âm của xác suất đầu ra tương ứng với lớp chính xác. Do đó, chúng ta cũng gọi đây là hàm mất mát log hợp lý âm (negative log likelihood loss):

$$L_{CE}(\hat{y}, \mathbf{y}) = -\log \hat{y}_c \quad (\text{với } c \text{ là lớp chính xác}) \tag{1.25}$$

Thay thế công thức softmax với K là số lượng lớp:

$$L_{CE}(\hat{y}, \mathbf{y}) = -\log \frac{\exp(z_c)}{\sum_{j=1}^{K} \exp(z_j)} \quad (\text{với } c \text{ là lớp chính xác}) \tag{1.26}$$

Hàm mất mát này chỉ phụ thuộc vào xác suất dự đoán $\hat{y}_c$ của lớp đúng c.

- Công thức (1.26) biểu diễn mất mát thông qua đầu vào z_j của lớp softmax trước khi áp dụng hàm mũ. Mục tiêu tối ưu là tăng xác suất $\hat{y}_c$ (giảm L_{CE}) bằng cách điều chỉnh các giá trị z_j.

Thuật toán tối ưu

Algorithm 1 Thuật toán Gradient_descent_incremental(D, η)

Khởi tạo w ($w_i \leftarrow$ là một giá trị ngẫu nhiên nhỏ)
repeat
  for Với mỗi mẫu huấn luyện $(x, d) \in D$ **do**
    Tính giá trị đầu ra của mạng
    for mỗi trọng số w_i **do**
      $w_i \leftarrow w_i - \eta \left(\frac{\partial E_x}{\partial w_i} \right)$
    end for
  end for
until dừng khi điều kiện thỏa mãn
return w

Giải thuật học lan truyền ngược tìm kiếm một vectơ các trọng số (weights vector) giúp cực tiểu hóa lỗi tổng thể của hệ thống đối với tập học. Giải thuật lan truyền ngược bao gồm 2 giai đoạn (bước) [3]:

1. **Giai đoạn lan truyền tiến**: Dữ liệu đầu vào (biểu diễn dưới dạng vector) được truyền tuần tự qua mạng theo chiều từ tầng đầu vào, qua các tầng ẩn trung gian, và cuối cùng đạt đến tầng đầu ra.

2. **Giai đoạn lan truyền ngược lỗi (Error backward)**: Từ kết quả đầu ra mong muốn của vector đầu vào, hệ thống xác định sai số hiện tại. Quá trình này bắt đầu tại tầng đầu ra, nơi sai số được truyền ngược qua cấu trúc mạng theo hướng từ tầng sau về tầng trước (ngược chiều với quá trình lan truyền thuận). Cụ thể, việc lan truyền ngược này thực hiện tính toán gradient cục bộ (theo phương pháp truy hồi) cho từng đơn vị nơ-ron trong mạng.

1.2.5 Một số chú ý trong mạng nơ ron

Khởi tạo giá trị của các trọng số cho mạng nơ ron:

Khởi tạo trọng số là yếu tố quan trọng quyết định hiệu quả huấn luyện mạng nơ-ron nhân tạo. Phương pháp khởi tạo không chỉ ảnh hưởng đến tốc độ hội tụ mà còn quyết định chất lượng mô hình cuối cùng. Khi các trọng số được khởi tạo với giá trị quá lớn, hệ thống thường gặp phải tình trạng các hàm sigmoid bão hòa sớm hoặc mắc kẹt tại các điểm yên ngựa trong không gian tối ưu hóa. Để khắc phục vấn đề này, các nhà nghiên cứu thường áp dụng phương pháp khởi tạo ngẫu nhiên với giá trị nhỏ thông qua các phân phối xác suất như phân phối chuẩn Gaussian hay phân phối đều Uniform. Cách tiếp cận này giúp duy trì sự ổn định trong quá trình lan truyền ngược và đảm bảo mô hình có thể học các đặc trưng một cách hiệu quả.

Tốc độ học (learning rate) η: Tốc độ học (learning rate) đóng vai trò then chốt trong quá trình huấn luyện các mô hình học máy, quyết định độ lớn của mỗi bước điều chỉnh trọng số khi tối ưu hóa hàm mất mát. Tham số này không chỉ tác động trực tiếp đến tốc độ hội tụ mà còn ảnh hưởng đáng kể đến hiệu suất cuối cùng của mô hình, khiến việc lựa chọn giá trị phù hợp trở thành yếu tố quan trọng trong quá trình điều chỉnh tham số.

Khi tốc độ học η được thiết lập quá cao, quá trình huấn luyện có thể hội tụ nhanh chóng nhưng đồng thời dễ bỏ lỡ điểm tối ưu toàn cục hoặc mắc kẹt tại các điểm yên ngựa (saddle points). Ngược lại, tốc độ học quá thấp tuy đảm bảo tính ổn định nhưng lại khiến thời gian huấn luyện kéo dài đáng kể.

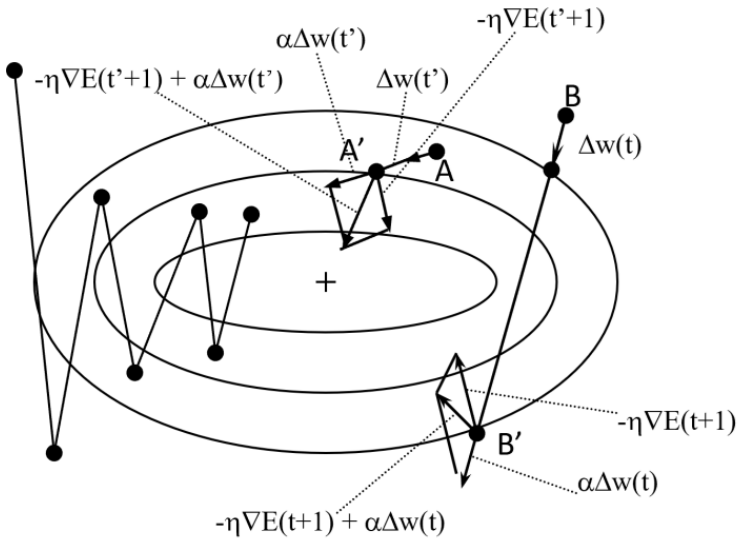
Trong thực tế, việc xác định tốc độ học tối ưu thường được thực hiện thông qua thử nghiệm (experimentally) cho từng bài toán cụ thể. Đáng lưu ý, giá trị phù hợp ở giai đoạn đầu huấn luyện có thể không còn hiệu quả ở các giai đoạn sau. Do đó, các phương pháp tốc độ học thích nghi (adaptive learning rate) hoặc lập lịch giảm dần (learning rate scheduling) thường được áp dụng để tự động điều chỉnh η theo tiến trình huấn luyện, nhằm cân bằng giữa tốc độ hội tụ và chất lượng mô hình cuối cùng.

Momentum: Phương pháp Gradient descent có thể rất chậm nếu η nhỏ, và có thể dao động mạnh nếu η quá lớn. Để giảm mức độ dao động, cần đưa vào một thành phần momentum (hình 1.10).

Công thức cập nhật trọng số với momentum:

$$\Delta w(t) = -\eta \nabla E(t) + \alpha \Delta w(t-1) \tag{1.27}$$

trong đó $\alpha \in [0, 1]$ là một tham số momentum (thường lấy $\alpha = 0.9$). Dựa trên các kinh nghiệm, ta nên chọn các giá trị hợp lý cho tốc độ học η và momentum α thoả mãn $(\eta + \alpha) \geq 1$ trong đó $\alpha > \eta$ để tránh dao động.



Hình 1.10: Hình minh họa quá trình cập nhật tham số dựa trên momentum. Quỹ đạo bên trái không sử dụng momentum. Quỹ đạo bên phải có sử dụng momentum.

Số Lượng Nơ-ron ở Tầng Ẩn

Việc xác định số lượng nơ-ron trong tầng ẩn là một vấn đề then chốt khi triển khai mạng nơ-ron truyền thẳng đa tầng để giải quyết các bài toán thực tiễn. Trên thực tế, không có phương pháp chuẩn nào để tính toán chính xác số nơ-ron tối ưu nhằm đạt được độ chính xác mong muốn cho hệ thống. Thông thường, kích thước tầng ẩn được xác định thông qua quá trình thử nghiệm và điều chỉnh dựa trên kết quả thực nghiệm.

1.3 Giới thiệu về học sâu

1.3.1 Tổng quan

Mô hình deep learning - học sâu là một nhánh nhỏ của AI - artificial intelligence (trí tuệ nhân tạo). Nó hoạt động dựa trên mạng nơ-ron nhân tạo (neural networks) để phân tích, xử lý dữ liệu và mô phỏng bộ não của con người.

Deep Learning (học sâu) cũng có thể được coi là một lĩnh vực thuộc Machine Learning – nơi máy tính tự học và cải thiện thông qua các thuật toán. Thế nhưng, Deep Learning được xây dựng dựa trên những khái niệm phức tạp hơn.

Chủ yếu hoạt động với mạng thần kinh nhân tạo để bắt chước khả năng tư duy và suy nghĩ của bộ não con người. Thực ra các khái niệm liên quan đến mạng nơ-ron nhân tạo và Deep Learning đã được phát triển những năm 1960. Nhưng nó bị giới hạn bởi lượng dữ liệu và khả năng tính toán tại thời điểm đó. Trong những năm gần đây, những tiến bộ trong phân tích big data đã cho phép chúng ta tận dụng tối đa khả năng của mạng lưới thần kinh nhân tạo. Mạng nơ-ron nhân tạo (neural networks) là động lực chính đằng sau sự phát triển của Deep Learning.

Mạng lưới thần kinh sâu (DNN - Deep neural networks) bao gồm nhiều lớp tế bào thần kinh khác nhau, có khả năng thực hiện các tính toán rất phức tạp. Deep Learning hiện đang phát triển rất nhanh và được coi là một trong những bước đột phá lớn nhất trong Machine Learning.

Bộ não con người chính là nguồn cảm hứng cho kiến trúc mạng nơ-ron. Các tế bào não của con người, còn được gọi là nơ-ron, tạo thành một mạng lưới phức tạp, có tính liên kết cao và gửi các tín hiệu điện đến nhau để giúp con người xử lý thông tin. Tương tự, một mạng nơ-ron nhân tạo được tạo ra từ các tế bào nơ-ron nhân tạo, cùng nhau phối hợp để giải quyết một vấn đề. Nơ-ron nhân tạo là các mô-đun phần mềm, được gọi là nút và mạng nơ-ron nhân tạo là các chương trình phần mềm hoặc thuật toán mà về cơ bản, sử dụng hệ thống máy tính để giải quyết các phép toán.

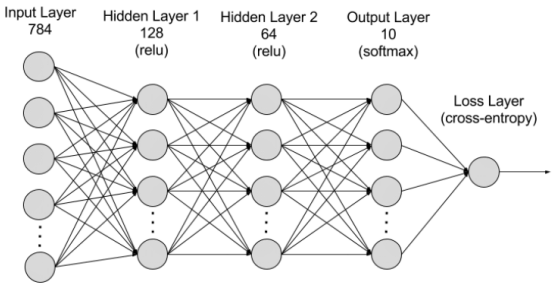
Một mạng nơ-ron cơ bản bao gồm các nơ-ron nhân tạo liên kết theo 3 lớp:

Lớp đầu vào: Thông tin từ thế giới bên ngoài đi vào mạng nơ-ron nhân tạo qua lớp đầu vào. Các nút đầu vào xử lý dữ liệu, phân tích hoặc phân loại và sau đó chuyển dữ liệu sang lớp tiếp theo.

Lớp ẩn: Dữ liệu đi vào lớp ẩn đến từ lớp đầu vào hoặc các lớp ẩn khác. Mạng nơ-ron nhân tạo có thể có một số lượng lớn lớp ẩn. Mỗi lớp ẩn phân tích dữ liệu đầu ra từ lớp trước, xử lý dữ liệu đó sâu hơn và rồi chuyển dữ liệu sang lớp tiếp theo.

Lớp đầu ra: Lớp đầu ra cho ra kết quả cuối cùng của tất cả dữ liệu được xử lý bởi mạng nơ-ron nhân tạo. Lớp này có thể có một hoặc nhiều nút. Ví dụ: giả sử chúng ta gặp phải một vấn đề phân loại nhị phân (có/không), lớp đầu ra sẽ có một nút đầu ra, nút này sẽ cho kết quả 1 hoặc 0. Tuy nhiên, nếu chúng ta gặp phải vấn đề phân loại nhiều lớp, lớp đầu ra sẽ có thể bao gồm nhiều hơn một nút đầu ra.

Kiến trúc mạng nơ-ron chuyên sâu: Mạng nơ-ron chuyên sâu, hoặc mạng deep learning, có nhiều lớp ẩn với hàng triệu nơ-ron nhân tạo liên kết với nhau. Một con số, có tên gọi là trọng số, đại diện cho các kết nối giữa hai nút. Trọng số sẽ dương nếu một nút kích thích nút còn lại, hoặc âm nếu một nút ngăn cản nút còn lại. Các nút với trọng số cao hơn sẽ có ảnh hưởng lớn hơn lên các nút khác. Về mặt lý thuyết, mạng nơ-ron chuyên sâu có thể ánh xạ bất kỳ loại dữ liệu đầu vào với bất kỳ loại dữ liệu đầu ra nào. Tuy nhiên, chúng cũng cần được huấn luyện nhiều hơn so với các phương pháp máy học khác. Chúng cần hàng triệu ví dụ về dữ liệu đào tạo thay vì hàng trăm hoặc hàng nghìn ví dụ mà một mạng đơn giản hơn thường cần.



Hình 1.11: Kiến trúc mạng nơ ron chuyên sâu

Mạng nơ-ron nhân tạo có thể được phân loại theo phương thức dữ liệu được truyền từ nút đầu vào đến nút đầu ra. Dưới đây là một số ví dụ:

Mạng nơ-ron truyền thẳng: Mạng nơ-ron truyền thẳng xử lý dữ liệu theo một chiều, từ nút đầu vào đến nút đầu ra. Mỗi nút trong một lớp được kết nối với tất cả các nút trong lớp tiếp theo. Mạng truyền thẳng sử dụng một quy trình phản hồi để cải thiện dự đoán theo thời gian.

Thuật toán truyền ngược: Mạng nơ-ron nhân tạo liên tục học hỏi bằng cách sử dụng vòng lặp phản hồi hiệu chỉnh để cải thiện phân tích dự đoán của chúng. Đơn giản mà nói, bạn có thể coi rằng dữ liệu truyền từ nút đầu vào đến nút đầu ra qua nhiều lối đi khác nhau trong mạng nơ-ron. Chỉ có duy nhất một lối đi chính xác, ánh xạ nút đầu vào đến nút đầu ra thích hợp. Để tìm ra lối đi này, mạng nơ-ron sử dụng một vòng lặp phản hồi với cách thức hoạt động như sau:

- Mỗi nút đưa ra một dự đoán về nút tiếp theo trên lối đi.

- Nút này sẽ kiểm tra tính chính xác của dự đoán. Các nút sẽ chỉ định giá trị trọng số cao hơn cho những lối đi tới nhiều dự đoán chính xác hơn và giá trị trọng số thấp hơn cho các lối đi tới dự đoán không chính xác.

- Đối với điểm dữ liệu tiếp theo, các nút đưa ra dự đoán mới bằng cách sử dụng các lối đi có trọng số cao hơn rồi lặp lại Bước 1.

Mạng nơ-ron tích chập: Những lớp ẩn trong mạng nơ-ron tích chập thực hiện các chức năng toán học cụ thể, như tóm tắt hoặc sàng lọc, được gọi là tích chập. Chúng rất hữu ích trong việc phân loại hình ảnh vì chúng có thể trích xuất các đặc điểm liên quan từ hình ảnh, điều này có lợi cho việc nhận dạng và phân loại hình ảnh. Biểu mẫu mới dễ xử lý hơn mà không làm mất đi các đặc điểm quan trọng để đưa ra dự đoán chính xác. Mỗi lớp ẩn trích xuất và xử lý các đặc điểm hình ảnh khác nhau, như các cạnh, màu sắc và độ sâu.

Mạng nơ-ron hồi quy:

RNN được thiết kế để xử lý dữ liệu chuỗi thời gian, nơi mỗi đầu ra phụ thuộc vào các trạng thái trước đó. Điều này được thực hiện thông qua việc duy trì một trạng thái ẩn (hidden state) để lưu trữ thông tin ngữ cảnh.

1.3.2 Một số ứng dụng của học sâu

Xe tự lái: Một trong những ứng dụng đáng chú ý nhất của Deep Learning trong đời sống hàng ngày là công nghệ xe tự lái. Được sử dụng rộng rãi trong lĩnh vực này, Deep Learning giúp xe tự động nhận diện và hiểu môi trường xung quanh bằng cách sử dụng mạng nơ-ron phức tạp. Các mô hình Deep Learning có khả năng xử lý hình ảnh từ các camera và cảm biến để nhận biết các đối tượng, tính toán khoảng cách, phân loại làn đường, xác định tín hiệu giao thông và dự đoán hành vi của các phương tiện khác. Thông qua việc xử lý lượng lớn dữ liệu và áp dụng thuật toán học sâu, các hãng xe tiên phong như Tesla đã đạt được những tiến bộ đáng kể trong việc phát triển công nghệ xe tự lái.

Nhận dạng giọng nói: Deep Learning đã mang lại sự tiến bộ đáng kể trong việc nhận dạng và hiểu giọng nói trong cuộc sống hàng ngày. Các mô hình Deep Learning được áp dụng trong các trợ lý ảo như Siri, Google Assistant và Alexa, giúp chúng có khả năng hiểu và thực hiện các lệnh dựa trên giọng nói của người dùng. Bằng cách sử dụng mạng nơ-ron phức tạp, Deep Learning có khả năng phân tích âm thanh, xử lý ngôn ngữ tự nhiên và tìm hiểu ý nghĩa của câu nói. Điều này cho phép trợ lý ảo nhận diện và thực hiện các tác vụ như gọi điện, tìm kiếm thông tin, đặt lịch hẹn và điều khiển các thiết bị trong nhà chỉ bằng giọng nói. Sự phát triển của Deep Learning trong lĩnh vực này đã mang đến một trải nghiệm tương tác người - máy tiện lợi và tự nhiên hơn.

Nhận dạng ảnh: Deep Learning đã có những đóng góp đáng kể trong việc nhận dạng và xử lý ảnh trong cuộc sống hàng ngày. Áp dụng Deep Learning vào lĩnh vực này, chúng ta đã có khả năng phân loại, nhận biết và hiểu nội dung của các hình ảnh. Các mô hình Deep Learning sử dụng mạng nơ-ron sâu để học và nhận diện các đặc trưng trong ảnh, từ đó đưa ra những dự đoán chính xác về nội dung của ảnh. Ví dụ, Deep Learning có thể được sử dụng để gắn nhãn tự động cho ảnh, nhận diện khuôn mặt, phân loại đối tượng, xử lý ảnh y tế, và nhiều ứng dụng khác. Công nghệ này đã mang lại tiện ích lớn cho việc tìm kiếm và quản lý ảnh số, cải thiện trải nghiệm người dùng trong các ứng dụng mạng xã hội, hỗ trợ trong nhiều lĩnh vực khoa học và y tế.

Tự động gợi ý và cá nhân hóa:

Deep Learning đã đóng góp quan trọng trong việc tạo ra các hệ thống gợi ý và cá nhân hóa trong cuộc sống hàng ngày. Các mô hình Deep Learning được sử dụng để phân tích và hiểu hành vi người dùng, từ đó đưa ra các đề xuất và gợi ý phù hợp với sở thích cá nhân. Ví dụ, các nền tảng như Netflix, Spotify và Amazon sử dụng Deep Learning để phân tích lịch sử xem, lắng nghe và mua sắm của người dùng. Dựa trên thông tin này, họ có thể đề xuất nội dung, bài hát, phim, sách và sản phẩm khác mà người dùng có thể quan tâm. Công nghệ Deep Learning đã cải thiện trải nghiệm người dùng, mang lại sự tiện ích và tạo ra cá nhân hóa trong các dịch vụ và sản phẩm mà chúng ta sử dụng hàng ngày.

Dịch máy:

Deep Learning đã có những đóng góp quan trọng trong lĩnh vực dịch máy tự động, làm cho việc diễn giải và chuyển đổi giữa các ngôn ngữ trở nên hiệu quả hơn. Các mô hình Deep Learning sử dụng mạng nơ-ron sâu để học và hiểu cấu trúc ngôn ngữ, cùng với việc áp dụng các phương pháp xử lý ngôn ngữ tự nhiên. Kết quả là, chúng ta có khả năng dịch văn bản tự động với độ chính xác và dịch thuật gần với ngôn ngữ tự nhiên hơn. Deep Learning đã giúp cải thiện các công cụ dịch trực tuyến, ứng dụng di động và hệ thống dịch máy tự động, đồng thời tiếp cận và giao tiếp với người nói ngôn ngữ khác nhau trở nên dễ dàng hơn và thuận tiện hơn.

Phân tích cảm xúc: Lĩnh vực phân tích cảm xúc là một ứng dụng quan trọng của Deep Learning trong việc hiểu và đánh giá cảm xúc của con người thông qua văn bản, bình luận, đánh giá, và các dữ liệu ngôn ngữ khác. Công nghệ Deep Learning có thể được áp dụng để xử lý và phân tích lượng lớn dữ liệu ngôn ngữ tự nhiên, từ đó nhận biết và phân loại cảm xúc như vui vẻ, buồn bã, hài lòng hay không hài lòng. Các công ty và tổ chức có thể tận dụng Deep Learning để phân tích đánh giá, bình luận trên mạng xã hội, tweet, thông tin phản hồi từ khách hàng và nguồn dữ liệu khác. Qua đó, họ có thể hiểu được ý kiến, nhận định và cảm xúc của khách hàng đối với sản phẩm, dịch vụ, hoặc sự kiện cụ thể. Kết

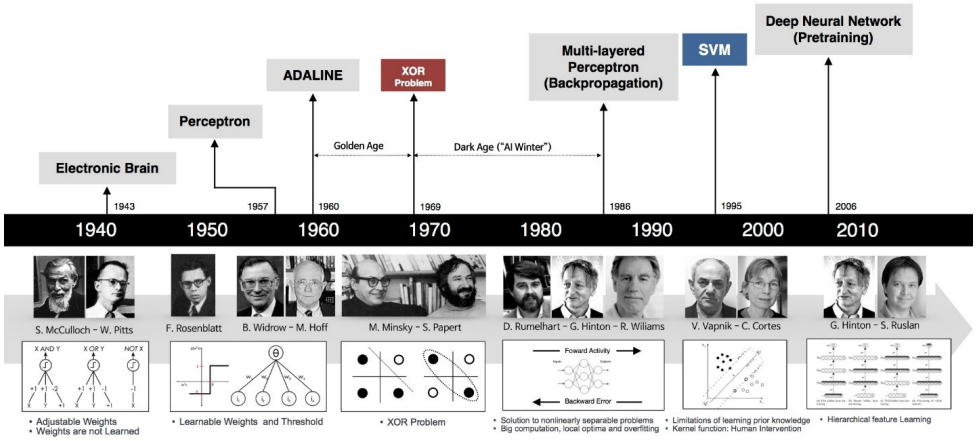
quả phân tích cảm xúc có thể cung cấp thông tin quý giá để hỗ trợ quyết định kinh doanh, phát triển chiến lược Marketing, cải thiện chất lượng sản phẩm và tương tác với khách hàng một cách tốt nhất.

Mạng xã hội: Deep Learning được sử dụng trong các nền tảng mạng xã hội như X, Instagram và Facebook để cải thiện trải nghiệm người dùng. Công nghệ này giúp phân tích dữ liệu lớn và hiểu sở thích của người dùng, đồng thời ngăn chặn nội dung bạo lực và không phù hợp. Deep Learning cũng được áp dụng để gợi ý nội dung, bạn bè và dịch vụ phù hợp, cùng nhận diện khuôn mặt trong ảnh.

Lĩnh vực y tế: Deep Learning đã đóng góp đáng kể trong lĩnh vực y tế, đặc biệt là trong việc dự đoán tình trạng bệnh, chẩn đoán ung thư và phân tích kết quả hình ảnh y tế như MRI và X-ray. Các mô hình Deep Learning được huấn luyện trên dữ liệu lớn từ hàng ngàn bệnh nhân, giúp xác định các mẫu và đặc trưng quan trọng trong dữ liệu y tế. Điều này cho phép họ tạo ra các mô hình dự đoán tình trạng bệnh, từ việc ước lượng nguy cơ mắc bệnh đến dự đoán kết quả điều trị. Ngoài ra, Deep Learning cũng được sử dụng trong quá trình chẩn đoán ung thư. Các mô hình Deep Learning có khả năng phân tích và nhận dạng các đặc điểm bất thường trong hình ảnh y tế, giúp phát hiện sớm các dấu hiệu của ung thư và hỗ trợ các chuyên gia y tế trong quá trình chẩn đoán.

1.3.3 Lịch sử phát triển của học sâu

Deep learning được nhắc đến nhiều trong những năm gần đây, nhưng những nền tảng cơ bản đã xuất hiện từ rất lâu. Chúng ta cùng quan sát hình dưới đây: Perceptron (60s)



Hình 1.12: Lịch sử của học sâu

Một trong những nền móng đầu tiên của neural network và deep learning là perceptron learning algorithm (hoặc gọn là perceptron). Perceptron là một thuật toán supervised learning giúp giải quyết bài toán phân lớp nhị phân, được khởi nguồn bởi Frank Rosenblatt năm 1957 trong một nghiên cứu được tài trợ bởi Văn phòng nghiên cứu hải quân Hoa Kỳ (U.S Office of Naval Research – từ một cơ quan liên quan đến quân sự). Thuật toán perceptron được chứng minh là hội tụ nếu hai lớp dữ liệu là linearly separable. Với thành công này, năm 1958, trong một hội thảo, Rosenblatt đã có một phát biểu gây tranh cãi. Từ phát biểu này, tờ New York Times đã có một bài báo cho rằng perceptron được Hải quân Hoa Kỳ mong đợi "có thể đi, nói chuyện, nhìn, viết, tự sinh sản, và tự nhận thức được sự tồn tại của mình". (Chúng ta biết rằng cho tới giờ các hệ thống nâng cao hơn perceptron nhiều lần vẫn chưa thể).

Mặc dù thuật toán này mang lại nhiều kỳ vọng, nó nhanh chóng được chứng minh không thể giải quyết những bài toán đơn giản. Năm 1969, Marvin Minsky và Seymour Papert trong cuốn sách nổi tiếng Perceptrons đã chứng minh rằng không thể 'học' được hàm số XOR khi sử dụng perceptron. Phát hiện này làm choáng váng giới khoa học thời gian đó (bây giờ chúng ta thấy việc này khá hiển nhiên). Perceptron được chứng minh rằng chỉ hoạt động nếu dữ liệu là linearly separable.

Phát hiện này khiến cho các nghiên cứu về perceptron bị gián đoạn gần 20 năm. Thời kỳ này còn được gọi là Mùa đông AI thứ nhất (The First AI winter).

Cho tới khi MLP và Backpropagation ra đời (80s).

Geoffrey Hinton tốt nghiệp PhD ngành neural networks năm 1978. Năm 1986, ông cùng với hai tác giả khác xuất bản một bài báo khoa học trên Nature với tựa đề "Learning representations by back-propagating errors". Trong bài báo này, nhóm của ông chứng minh rằng neural nets với nhiều hidden layer (được gọi là multi-layer perceptron hoặc MLP) có thể được huấn luyện một cách hiệu quả dựa trên một quy trình đơn giản được gọi là backpropagation (backpropagation là tên gọi mỹ miều của quy tắc chuỗi – chain rule – trong tính đạo hàm. Việc tính được đạo hàm của hàm số phức tạp mô tả quan hệ giữa đầu vào và đầu ra của một neural net là rất quan trọng vì hầu hết các thuật toán tối ưu đều được

thực hiện thông qua việc tính đạo hàm, gradient descent là một ví dụ). Việc này giúp neural nets thoát được những hạn chế của perceptron về việc chỉ biểu diễn được các quan hệ tuyến tính. Để biểu diễn các quan hệ phi tuyến, phía sau mỗi layer là một hàm kích hoạt phi tuyến, ví dụ hàm sigmoid hoặc tanh. (ReLU ra đời năm 2012). Với hidden layers, neural nets được chứng minh rằng có khả năng xấp xỉ hầu hết bất kỳ hàm số nào qua một định lý được gọi là universal approximation theorem. Neurel nets quay trở lại cuộc chơi.

Thuật toán này mang lại một vài thành công ban đầu, nổi trội là convolutional neural nets (convnets hay CNN) (còn được gọi là LeNet) cho bài toán nhận dạng chữ số viết tay được khởi nguồn bởi Yann LeCun tại AT&T Bell Labs (Yann LeCun là sinh viên sau cao học của Hinton tại đại học Toronto năm 1987-1988). Dưới đây là bản demo được lấy từ trang web của LeNet, network là một CNN với 5 layer, còn được gọi là LeNet-5 (1998).

Mô hình này được sử dụng rộng rãi trong các hệ thống đọc số viết tay trên các check (séc ngân hàng) và mã vùng bưu điện của nước Mỹ.

LeNet là thuật toán tốt nhất thời gian đó cho bài toán nhận dạng ảnh chữ số viết tay. Nó tốt hơn MLP thông thường (với fully connected layer) vì nó có khả năng trích xuất được đặc trưng trong không gian hai chiều của ảnh thông qua các filters (bộ lọc) hai chiều. Hơn nữa, các filter này nhỏ nên việc lưu trữ và tính toán cũng tốt hơn so với MLP thông thường. (Yan LeCun có xuất phát từ Electrical Engineering nên rất quen thuộc với các bộ lọc.)

Mùa đông AI thứ hai (90s - đầu 2000s)

Các mô hình tương tự được kỳ vọng sẽ giải quyết nhiều bài toán image classification khác. Tuy nhiên, không như các chữ số, các loại ảnh khác lại rất hạn chế vì máy ảnh số chưa phổ biến tại thời điểm đó. Ảnh được gán nhãn lại càng hiếm. Trong khi để có thể huấn luyện được mô hình convnets, ta cần rất nhiều dữ liệu huấn luyện. Ngay cả khi dữ liệu có đủ, một vấn đề nan giải khác là khả năng tính toán của các máy tính thời đó còn rất hạn chế.

Một hạn chế khác của các kiến trúc MLP nói chung là hàm mất mát không phải là một hàm lồi. Việc này khiến cho việc tìm nghiệm tối ưu toàn cục cho bài toán tối ưu hàm mất mát trở nên rất khó khăn. Một vấn đề khác liên quan đến giới hạn tính toán của máy tính cũng khiến cho việc huấn luyện MLP không hiệu quả khi số lượng hidden layers lớn lên. Vấn đề này có tên là vanishing gradient.

Nhắc lại rằng hàm kích hoạt được sử dụng thời gian đó là sigmoid hoặc tanh – là các hàm bị chặn trong khoảng (0, 1) hoặc (-1, 1) (Nhắc lại đạo hàm của hàm sigmoid là tích của hai số nhỏ hơn 1). Khi sử dụng backpropagation để tính đạo hàm cho các ma trận hệ số ở các lớp đầu tiên, ta cần phải nhân rất nhiều các giá trị nhỏ hơn 1 với nhau. Việc này khiến cho nhiều đạo hàm thành phần bằng 0 do xấp xỉ tính toán. Khi đạo hàm của một thành phần bằng 0, nó sẽ không được cập nhật thông qua gradient descent!

Những hạn chế này khiến cho neural nets một lần nữa rơi vào thời kỳ băng giá. Vào thời điểm những năm 1990 và đầu những năm 2000, neural nets dần được thay thế bởi support vector machines –SVM. SVMs có ưu điểm là bài toán tối ưu để tìm các tham số của nó là một bài toán lồi – có nhiều các thuật toán tối ưu hiệu quả giúp tìm nghiệm của nó. Các kỹ thuật về kernel cũng phát triển giúp SVMs giải quyết được cả các vấn đề về việc dữ liệu không phân biệt tuyến tính.

Nhiều nhà khoa học làm machine learning chuyển sang nghiên cứu SVM trong thời gian đó, trừ một vài nhà khoa học cứng đầu

Cái tên được làm mới – Deep Learning (2006)

Năm 2006, Hinton một lần nữa cho rằng ông biết bộ não hoạt động như thế nào, và giới thiệu ý tưởng của tiền huấn luyện không giám sát (unsupervised pretraining) thông qua deep belief nets (DBN). DBN có thể được xem như sự xếp chồng các unsupervised networks đơn giản như restricted Boltzman machine hay autoencoders.

Lấy ví dụ với autoencoder. Mỗi autoencoder là một neural net với một hidden layer. Số hidden unit ít hơn số input unit, và số output unit bằng với số input unit. Network này đơn giản được huấn luyện để kết quả ở output layer giống với kết quả ở input layer (và vì vậy được gọi là autoencoder). Quá trình dữ liệu đi từ input layer tới hidden layer có thể coi là mã hoá, quá trình dữ liệu đi từ hidden layer ra output layer có thể được coi là giải mã. Khi output giống với input, ta có thể thấy rằng hidden layer với ít unit hơn có để mã hoá input khá thành công, và có thể được coi mang những tính chất của input. Nếu ta bỏ output layer, cố định (freeze) kết nối giữa input và hidden layer, coi đầu ra của hidden layer là một input mới, sau đó huấn luyện một autoencoder khác, ta được thêm một hidden layer nữa. Quá trình này tiếp tục kéo dài ta sẽ được một network đủ sâu mà output của network lớn này (chính là hidden layer của autoencoder cuối cùng) mang thông tin của input ban đầu. Sau đó ta có thể thêm các layer khác tuỳ thuộc vào bài toán (chẳng hạn thêm softmax layer ở cuối cho bài toán classification). Cả network được huấn luyện thêm một vài epoch nữa. Quá trình này được gọi là tinh chỉnh (fine tuning).

Tại sao quá trình huấn luyện như trên mang lại nhiều lợi ích? Một trong những hạn chế đã đề cập của MLP là vấn đề vanishing gradient. Những ma trận trọng số ứng với các layer đầu của network rất

khó được huấn luyện vì đạo hàm của hàm mất mát theo các ma trận này nhỏ. Với ý tưởng của DBN, các ma trận trọng số ở những hidden layer đầu tiên được tiền huấn luyện (pretrained). Các trọng số được tiền huấn luyện này có thể coi là giá trị khởi tạo tốt cho các hidden layer phía đầu. Việc này giúp phần nào tránh được sự phiền hà của vanishing gradient.

Kể từ đây, neural networks với nhiều hidden layer được đổi tên thành deep learning. Vấn đề vanishing gradient được giải quyết phần nào (vẫn chưa thực sự triệt để), nhưng vẫn còn những vấn đề khác của deep learning: dữ liệu huấn luyện quá ít, và khả năng tính toán của CPU còn rất hạn chế trong việc huấn luyện các deep networks.

Đột phá (2012)

Năm 2012, cũng tại ILSVRC, Alex Krizhevsky, Ilya Sutskever, và Geoffrey Hinton (lại là ông) tham gia và đạt kết quả top-5 error rate 16%. Kết quả này làm sững sờ giới nghiên cứu thời gian đó. Mô hình là một Deep Convolutional Neural Network, sau này được gọi là AlexNet.

Trong bài báo này, rất nhiều các kỹ thuật mới được giới thiệu. Trong đó hai đóng góp nổi bật nhất là hàm ReLU và dropout. Hàm ReLU với cách tính và đạo hàm đơn giản (bằng 1 khi đầu vào không âm, bằng 0 khi ngược lại) giúp tốc độ huấn luyện tăng lên đáng kể. Ngoài ra, việc ReLU không bị chặn trên bởi 1 (như softmax hay tanh) khiến cho vấn đề vanishing gradient cũng được giải quyết phần nào. Dropout cũng là một kỹ thuật đơn giản và cực kỳ hiệu quả. Trong quá trình training, nhiều hidden unit bị tắt ngẫu nhiên và mô hình được huấn luyện trên các bộ tham số còn lại. Trong quá trình test, toàn bộ các unit sẽ được sử dụng. Cách làm này khá là có lý khi đối chiếu với con người. Nếu chỉ dùng một phần năng lực đã đem lại hiệu quả thì dùng toàn bộ năng lực sẽ mang lại hiệu quả cao hơn. Việc này cũng giúp cho mô hình tránh được overfitting và cũng được coi giống với kỹ thuật ensemble trong các hệ thống machine learning khác. Với mỗi cách tắt các unit, ta có một mô hình khác nhau. Với nhiều tổ hợp unit bị tắt khác nhau, ta thu được nhiều mô hình. Việc kết hợp ở cuối cùng được coi như sự kết hợp của nhiều mô hình (và vì vậy, nó giống với ensemble learning).

Một trong những yếu tố quan trọng nhất giúp AlexNet thành công là việc sử dụng GPU (card đồ hoạ) để huấn luyện mô hình. GPU được tạo ra cho game thủ, với khả năng chạy song song nhiều lõi, đã trở thành một công cụ cực kỳ phù hợp với các thuật toán deep learning, giúp tăng tốc thuật toán lên nhiều lần so với CPU.

Sau AlexNet, tất cả các mô hình giành giải cao trong các năm tiếp theo đều là các deep networks (ZFNet 2013, GoogLeNet 2014, VGG 2014, ResNet 2015).

Những công ty công nghệ lớn cũng để ý tới việc phát triển các phòng nghiên cứu deep learning trong thời gian này. Rất nhiều các ứng dụng công nghệ đột phá đã được áp dụng vào cuộc sống hàng ngày. Cũng kể từ năm 2012, số lượng các bài báo khoa học về deep learning tăng lên theo hàm số mũ. Các blog về deep learning cũng tăng lên từng ngày.

Điều gì mang đến sự thành công của deep learning?

Rất nhiều những ý tưởng cơ bản của deep learning được đặt nền móng từ những năm 80-90 của thế kỷ trước, tuy nhiên deep learning chỉ đột phá trong khoảng 5-6 năm nay. Vì sao? Có nhiều nhân tố dẫn đến sự bùng nổ này:

- Sự ra đời của các bộ dữ liệu lớn được gán nhãn.
- Khả năng tính toán song song tốc độ cao của GPU.
- Sự ra đời của ReLU và các hàm kích hoạt liên quan làm hạn chế vấn đề vanishing gradient.
- Sự cải tiến của các kiến trúc: GoogLeNet, VGG, ResNet, ... và các kỹ thuật transfer learning, fine tuning.
- Nhiều kỹ thuật regularization mới: dropout, batch normalization, data augmentation.
- Nhiều thư viện mới hỗ trợ việc huấn luyện deep network với GPU: theano, caffe, mxnet, tensorflow, pytorch, keras, ...
- Nhiều kỹ thuật tối ưu mới: Adagrad, RMSProp, Adam, ...

1.4 Tổng quan về Pytorch

Trong thời đại công nghệ trí tuệ nhân tạo (AI) và học sâu (deep learning) phát triển mạnh mẽ, PyTorch đã trở thành một trong những thư viện nổi bật nhất, được sử dụng rộng rãi bởi các nhà nghiên cứu và kỹ sư AI trên toàn thế giới. Được giới thiệu lần đầu vào năm 2016 bởi Facebook's AI Research (FAIR), PyTorch nhanh chóng thu hút được sự quan tâm nhờ tính linh hoạt, giao diện dễ học, và hiệu suất vượt trội.

PyTorch cung cấp nhiều tính năng mạnh mẽ, hỗ trợ người dùng trong các bài toán học sâu và mô phỏng AI:

- Tensor Computing: PyTorch cung cấp khả năng xử lý tensor (mảng số đa chiều) tương tự như NumPy, nhưng với khả năng tính toán nhanh trên GPU.

- Dynamic Computational Graphs: Khác với các thư viện như TensorFlow (trước TensorFlow 2.0), PyTorch sử dụng biểu đồ tính toán linh động, cho phép thay đổi cấu trúc mô hình trong khi huấn luyện.
- Khả Năng Tích Hợp Mạnh Mẽ: Hỗ trợ tích hợp với các thư viện machine learning như scikit-learn và NumPy.
- Dễ Học: Giao diện API trực quan, người dùng dễ dàng hiểu và triển khai.
- Hỗ Trợ GPU: Tính toán nhanh trên GPU với CUDA của NVIDIA.

Để bắt đầu làm việc trên Pytorch, chúng ta import Pytorch. Phần giới thiệu về Pytorch cũng như các bài tập sau này chúng ta mô phỏng trên môi trường google colab.

Import Pytorch

```
import torch
import torch.nn as nn

import pprint
pp = pprint.PrettyPrinter()
```

1.4.1 Tensors

Tensor là khối xây dựng cơ bản nhất của PyTorch. Mỗi tensor là một ma trận đa chiều; ví dụ, một hình vuông 256x256 có thể được biểu diễn bằng một tensor 3x256x256, trong đó chiều đầu tiên biểu diễn màu sắc. Sau đây là cách tạo một tensor:

Biểu diễn danh sách các danh sách:

```
list_of_lists = [
  [1, 2, 3],
  [4, 5, 6],
]
print(list_of_lists)
```

Kết quả:

```
[[1, 2, 3], [4, 5, 6]]
```

Dùng tensors để thay thế:

```
data = torch.tensor(list_of_lists)
print(data)
```

Kết quả khi dùng tensors

```
tensor([[1, 2, 3],
        [4, 5, 6]])
```

Khởi tạo một tensors:

```
data = torch.tensor([
                      [0, 1],
                      [2, 3],
                      [4, 5]
                     ])
print(data)
```

Kết quả:

```
tensor([[0, 1],
        [2, 3],
        [4, 5]])
```

Mỗi tensor có một kiểu dữ liệu: các kiểu dữ liệu chính mà bạn cần quan tâm là số thực (torch.float32) và số nguyên (torch.int). Bạn có thể chỉ định kiểu dữ liệu một cách rõ ràng khi tạo tensor.

```
data = torch.tensor([
                      [0, 1],
                      [2, 3],
                      [4, 5]
                     ], dtype=torch.float32)
print(data)
```

Kết quả:

```
tensor([[0., 1.],
        [2., 3.],
        [4., 5.]])
```

Khởi tạo một tensor với một kiểu dữ liệu rõ ràng, Lưu ý các dấu chấm sau các số, chỉ ra rằng chúng là số thực:

```
data = torch.tensor([
                     [0.11111111, 1],
                     [2, 3],
                     [4, 5]
                    ])
print(data)
```

Kết quả:

```
tensor([[0.1111, 1.0000],
        [2.0000, 3.0000],
        [4.0000, 5.0000]])
```

Các hàm tiện ích cũng tồn tại để tạo ra các tensor có hình dạng và nội dung nhất định:

```
zeros = torch.zeros(2, 5)  # a tensor toan so 0
print(zeros)
```

Kết quả:

```
tensor([[0., 0., 0., 0., 0.],
        [0., 0., 0., 0., 0.]])
```

Tensor chứa toàn số 1:

```
ones = torch.ones(3, 4)
print(ones)
```

Kết quả:

```
tensor([[1., 1., 1., 1.],
        [1., 1., 1., 1.],
        [1., 1., 1., 1.]])
```

```
rr = torch.arange(1, 10) # range from [1, 10)
print(rr)
```

Kết quả:

```
tensor([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Phép toán trên tensor:

```
print(rr + 2)
print(rr * 2)
```

Kết quả:

```
tensor([ 3,  4,  5,  6,  7,  8,  9, 10, 11])
tensor([ 2,  4,  6,  8, 10, 12, 14, 16, 18])
```

```
a = torch.tensor([[1, 2], [2, 3], [4, 5]])      # (3, 2)
b = torch.tensor([[1, 2, 3, 4], [5, 6, 7, 8]])  # (2, 4)

print("A is", a)
print("B is", b)
print("The product is", a.matmul(b)) #(3, 4)
print("The other product is", a @ b) # +, -, *, @
```

Kết quả:

```
A is tensor([[1, 2],
        [2, 3],
        [4, 5]])
B is tensor([[1, 2, 3, 4],
        [5, 6, 7, 8]])
```

```
The product is tensor([[11, 14, 17, 20],
        [17, 22, 27, 32],
        [29, 38, 47, 56]])
The other product is tensor([[11, 14, 17, 20],
        [17, 22, 27, 32],
        [29, 38, 47, 56]])
```

Hình dạng của ma trận (có thể truy cập bằng .shape) được định nghĩa là kích thước của ma trận. Sau đây là một số ví dụ:

```
matr_2d = torch.tensor([[1, 2, 3], [4, 5, 6]])
print(matr_2d.shape)
print(matr_2d)
```

Kết quả:

```
torch.Size([2, 3])
tensor([[1, 2, 3],
        [4, 5, 6]])
```

Có thể sử dụng việc định hình lại tensor để thực hiện các thao tác hàng loạt dễ dàng hơn, nhưng hãy cẩn thận để dữ liệu được định hình lại theo thứ tự bạn mong đợi:

```
rr = torch.arange(1, 16)
print("So chieu hien tai:", rr.shape)
print("Noi dung hien tai", rr)
print()
rr = rr.view(5, 3)
print("Sau khi chinh lai chieu, Chieu hien tai:", rr.shape)
print("Noi dung hien tai:", rr)
```

Kết quả:

```
Chieu hien tai torch.Size([15])
Noi dung hien tai tensor([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15])

Sau khi chinh lai chieu, chieu hien tai torch.Size([5, 3])
Noi dung hien tai tensor([[ 1,  2,  3],
        [ 4,  5,  6],
        [ 7,  8,  9],
        [10, 11, 12],
        [13, 14, 15]])
```

Cuối cùng, chúng ta có thể chuyển đổi lẫn nhau các tensor với các mảng NumPy:

```
import numpy as np

# numpy.ndarray --> torch.Tensor:
arr = np.array([[1, 0, 5]])
data = torch.tensor(arr)
print("This is a torch.tensor", data)

# torch.Tensor --> numpy.ndarray:
new_arr = data.numpy()
print("This is a np.ndarray", new_arr)
```

Kết quả:

```
This is a torch.tensor tensor([[1, 0, 5]])
This is a np.ndarray [[1 0 5]]
```

Một trong những lý do chúng ta sử dụng tensor là các phép toán vectơ hóa: các phép toán được thực hiện song song trên một chiều cụ thể của tensor.

```
data = torch.arange(1, 36, dtype=torch.float32).reshape(5, 7)
print("Data is:", data)

# We can perform operations like *sum* over each row...
print("Taking the sum over rows:")
print(data.sum(dim=1)) #(5,)

# or over each column.
print("Taking thep sum over columns:")
print(data.sum(dim=0)) #(7,)

```

```
# Other operations are available:
print("Taking the stdev over rows:")
print(data.std(dim=1))
```

Kết quả:

```
Data is: tensor([[ 1.,  2.,  3.,  4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11., 12., 13., 14.],
        [15., 16., 17., 18., 19., 20., 21.],
        [22., 23., 24., 25., 26., 27., 28.],
        [29., 30., 31., 32., 33., 34., 35.]])
Taking the sum over rows:
tensor([ 28.,  77., 126., 175., 224.])
Taking thep sum over columns:
tensor([ 75.,  80.,  85.,  90.,  95., 100., 105.])
Taking the stdev over rows:
tensor([2.1602, 2.1602, 2.1602, 2.1602, 2.1602])
```

1.4.2 Indexing

Bạn có thể truy cập các phần tử tùy ý của một tensor bằng toán tử [].

```
# Initialize an example tensor
x = torch.Tensor([
                   [[1, 2], [3, 4]],
                   [[5, 6], [7, 8]],
                   [[9, 10], [11, 12]]
                  ])
x[0]
x[:, 0]
x[:, 0, 0]
```

Kết quả:

```
tensor([[1., 2.],
        [3., 4.]])

tensor([[ 1.,  2.],
        [ 5.,  6.],
        [ 9., 10.]])
tensor([1., 5., 9.])
```

Chúng ta cũng có thể truy cập các phần tử tùy ý trong mỗi chiều.

Ví dụ: Truy cập vào phần tử thứ 0 và thứ 1, mỗi phần tử hai lần, giống như xếp chồng x[0], x[0], x[1], x[1]

```
i = torch.tensor([0, 0, 1, 1])
x[i]
```

Kết quả:

```
tensor([[[1., 2.],
         [3., 4.]],

        [[1., 2.],
         [3., 4.]],

        [[5., 6.],
         [7., 8.]],

        [[5., 6.],
         [7., 8.]]])
```

Chúng ta hãy truy cập vào phần tử thứ 0 của phần tử thứ 1 và thứ 2

```
i = torch.tensor([1, 2])
j = torch.tensor([0])
x[i, j]
```

Kết quả:

```
tensor([[ 5.,  6.],
        [ 9., 10.]])
```

Chúng ta có thể lấy giá trị vô hướng từ một tensor bằng tem().

```
x[0, 0, 0]
x[0, 0, 0].item()
```

Kết quả:

```
tensor(1.)
1.0
```

1.4.3 Tự động tính đạo hàm

Pytorch nổi tiếng với tính năng phân biệt tự động. Chúng ta có thể gọi phương thức backward() để yêu cầu PyTorch tính toán các gradient, sau đó được lưu trữ trong thuộc tính grad.

```
x = torch.tensor([2.], requires_grad=True)
y = x * x * 3 # 3x^2
y.backward()
pp.pprint(x.grad)
```

Kết quả:

```
tensor([12.])
```

Chúng ta có thể thấy rằng x.grad được cập nhật thành tổng của các gradient đã tính toán cho đến nay. Khi chúng ta chạy backprop trong mạng nơ-ron, chúng ta sẽ tính tổng tất cả các gradient cho một nơ-ron cụ thể trước khi thực hiện cập nhật. Đây chính xác là những gì đang xảy ra ở đây! Đây cũng là lý do tại sao chúng ta cần chạy zero_grad() trong mọi lần lặp lại đào tạo (sẽ nói thêm về điều này sau). Nếu không, các gradient của chúng ta sẽ tiếp tục tích lũy từ lần lặp lại đào tạo này sang lần lặp lại đào tạo khác, điều này sẽ khiến các bản cập nhật của chúng ta bị sai.

```
x.grad = None
z = x * x * 3 # 3x^2
z.backward()
# y = x * x * 3
pp.pprint(x.grad)

z = x * x * 3 # 3x^2
z.backward()
# y = x * x * 3
pp.pprint(x.grad)

z = x * x * 3 # 3x^2
z.backward()
# y = x * x * 3
pp.pprint(x.grad)
```

Kết quả:

```
tensor([12.])
tensor([24.])
tensor([36.])
```

1.4.4 Mạng nơ ron

Phần trên, chúng ta đã biết sử dụng tensor, các thuộc tính của chúng và các phép toán cơ bản trên tensor. Phần này, chúng ta sẽ sử dụng các khối được xác định trước trong module torch.nn của PyTorch. Sau đó, chúng ta sẽ ghép các khối này lại với nhau để tạo ra các mạng phức tạp. Hãy bắt đầu bằng cách import module này với một bí danh để chúng ta không phải nhập torch mỗi khi sử dụng.

```
import torch.nn as nn
```

Chúng ta có thể sử dụng nn.Linear(H_in, H_out) để tạo một lớp tuyến tính. Điều này sẽ lấy một ma trận có (N, *, H_in) chiều và xuất ra một ma trận có (N, *, H_out). Dấu * biểu thị rằng có thể có số chiều tùy ý ở giữa. Lớp tuyến tính thực hiện phép toán Ax+b, trong đó A và b được khởi tạo ngẫu nhiên. Nếu chúng ta không muốn lớp tuyến tính học các tham số độ lệch, chúng ta có thể khởi tạo lớp của mình bằng độ lệch=False.

```
input = torch.ones(2,3,4)
# N* H_in -> N*H_out
linear = nn.Linear(4, 2)
linear_output = linear(input)
linear_output
linear_output.shape
list(linear.parameters()) # Ax + b
```

nn.Linear(4,2) có nghĩa là input 4 chiều và đầu ra của lớp mạng này là 2 chiều.
linear(input) là áp dụng lớp linear lên tensor input.
list(linear.parameters()): Liệt kê các tham số của lớp nn.Linear, bao gồm trọng số và bias.
Kết quả:

```
tensor([[[-0.8810,  0.3769],
         [-0.8810,  0.3769],
         [-0.8810,  0.3769]],

        [[-0.8810,  0.3769],
         [-0.8810,  0.3769],
         [-0.8810,  0.3769]]], grad_fn=<ViewBackward0>)

torch.Size([2, 3, 10, 11, 2])

[Parameter containing:
 tensor([[-0.1443,  0.2130,  0.2116, -0.4267],
         [-0.3379,  0.2243,  0.3289, -0.2484]], requires_grad=True),
 Parameter containing:
 tensor([-0.4608,  0.0073], requires_grad=True)]
```

1.4.5 Các lớp module khác

Có một số lớp được cấu hình sẵn khác trong module nn. Một số ví dụ thường được sử dụng là nn.Conv2d, nn.ConvTranspose2d, nn.BatchNorm1d, nn.BatchNorm2d, nn.Upsample và nn.MaxPool2d trong số nhiều lớp khác. Chúng ta sẽ tìm hiểu thêm về những lớp này khi chúng ta thực hiện các chương tiếp theo. Chúng ta sẽ cung cấp các kích thước cần thiết và PyTorch sẽ đảm nhiệm việc thiết lập chúng.

Lớp hàm kích hoạt: Chúng ta cũng có thể sử dụng module nn để áp dụng các hàm kích hoạt cho các tensor của mình. Các hàm kích hoạt được sử dụng để thêm tính phi tuyến tính vào mạng. Một số ví dụ về các hàm kích hoạt là nn.ReLU(), nn.Sigmoid() và nn.LeakyReLU().

```
sigmoid = nn.Sigmoid()
output = sigmoid(linear_output)
output
```

Kết quả:

```
tensor([[[0.2930, 0.5931],
         [0.2930, 0.5931],
         [0.2930, 0.5931]],

        [[0.2930, 0.5931],
         [0.2930, 0.5931],
         [0.2930, 0.5931]]], grad_fn=<SigmoidBackward0>)
```

Chúng ta có thể tạo các lớp và truyền đầu ra của một lớp làm đầu vào của lớp tiếp theo. Thay vì tạo các tensor trung gian chúng ta có thể sử dụng nn.

```
block = nn.Sequential(
    nn.Linear(4, 2),
    nn.Sigmoid()
)

input = torch.ones(2,3,4)
output = block(input)
output
```

Kết quả:

```
tensor([[[0.6822, 0.5056],
         [0.6822, 0.5056],
         [0.6822, 0.5056]],

```

```
        [[0.6822, 0.5056],
         [0.6822, 0.5056],
         [0.6822, 0.5056]]], grad_fn=<SigmoidBackward0>)
```

Thay vì sử dụng các module được xác định trước, chúng ta cũng có thể xây dựng module của riêng mình bằng cách mở rộng lớp nn.Module. Ví dụ, chúng ta có thể tự xây dựng nn.Linear (cũng mở rộng nn.Module) bằng cách sử dụng tensor đã giới thiệu trước đó! Chúng ta cũng có thể xây dựng các module mới, phức tạp hơn, chẳng hạn như mạng nơ ron tùy chỉnh. Để tạo một module tùy chỉnh, điều đầu tiên chúng ta phải làm là mở rộng nn.Module. Sau đó, chúng ta có thể khởi tạo các tham số của mình trong hàm __init__, bắt đầu bằng lệnh gọi đến hàm __init__ của siêu lớp. Tất cả các thuộc tính lớp mà chúng ta xác định là các đối tượng module nn đều được coi là tham số, có thể được học trong quá trình đào tạo. Tensor không phải là tham số, nhưng chúng có thể được chuyển thành tham số nếu chúng được gói trong lớp nn.Parameter.

Tất cả các lớp mở rộng nn.Module sẽ triển khai hàm forward(x), trong đó x là tensor. Đây là hàm được gọi khi một tham số được truyền vào module của chúng ta, chẳng hạn như trong model(x).

```
class MultilayerPerceptron(nn.Module):

  def __init__(self, input_size, hidden_size):
    super(MultilayerPerceptron, self).__init__()

    self.input_size = input_size
    self.hidden_size = hidden_size

    self.model = nn.Sequential(
        nn.Linear(self.input_size, self.hidden_size),
        nn.ReLU(),
        nn.Linear(self.hidden_size, self.input_size),
        nn.Sigmoid()
    )

  def forward(self, x):
    output = self.model(x)
    return output
```

Đây là một cách thay thế để định nghĩa cùng một lớp. Chúng ta có thể thay thế nn.Sequential bằng cách định nghĩa các lớp riêng lẻ trong phương thức __init__ và kết nối trong phương thức forward.

```
class MultilayerPerceptron(nn.Module):

  def __init__(self, input_size, hidden_size):
    super(MultilayerPerceptron, self).__init__()

    self.input_size = input_size
    self.hidden_size = hidden_size


    self.linear = nn.Linear(self.input_size, self.hidden_size)
    self.relu = nn.ReLU()
    self.linear2 = nn.Linear(self.hidden_size, self.input_size)
    self.sigmoid = nn.Sigmoid()

  def forward(self, x):
    linear = self.linear(x)
    relu = self.relu(linear)
    linear2 = self.linear2(relu)
    output = self.sigmoid(linear2)
    return output
```

Chúng ta đã định nghĩa lớp MultilayerPerceptron, tiếp theo chúng ta có thể khởi tạo nó và thực hiện như sau:

```
input = torch.randn(2, 5)
model = MultilayerPerceptron(5, 3)
model(input)
```

Kết quả:

```
tensor([[0.4621, 0.5310, 0.4358, 0.6027, 0.6664],
        [0.4217, 0.5628, 0.3208, 0.5732, 0.6326]], grad_fn=<SigmoidBackward0>)
```

Chúng ta có thể kiểm tra các tham số của mô hình bằng các phương thức named_parameters() và parameters().

```
list(model.named_parameters())
```

Kết quả:

```
[('linear.weight',
  Parameter containing:
  tensor([[-0.0359,  0.0409, -0.2182,  0.0993,  0.0216],
          [ 0.3572, -0.0213,  0.0229, -0.1651,  0.3042],
          [ 0.1404,  0.1992, -0.2556, -0.1605,  0.0395]], requires_grad=True)),
 ('linear.bias',
  Parameter containing:
  tensor([ 0.1759, -0.1290,  0.0760], requires_grad=True)),
 ('linear2.weight',
  Parameter containing:
  tensor([[ 0.2302,  0.0376, -0.4018],
          [-0.4662, -0.4792,  0.1319],
          [-0.5094,  0.3908, -0.1104],
          [ 0.3688,  0.1750, -0.3135],
          [ 0.3631,  0.3253, -0.2241]], requires_grad=True)),
 ('linear2.bias',
  Parameter containing:
  tensor([-0.1755,  0.3826, -0.4531,  0.3189,  0.5156], requires_grad=True))]
```

1.4.6 Tối ưu

Ở trên, chúng ta đã trình bày cách tính toán gradient bằng hàm backward(). Có gradient không đủ để các mô hình của chúng ta học được. Chúng ta cũng cần biết cách cập nhật các tham số của mô hình. Đây là lúc optomozers phát huy tác dụng. Module torch.optim chứa một số trình tối ưu hóa mà chúng ta có thể sử dụng. Một số ví dụ phổ biến là optim.SGD và optim.Adam. Khi khởi tạo trình tối ưu hóa, chúng ta truyền các tham số mô hình của mình, có thể truy cập bằng model.parameters(), cho trình tối ưu hóa biết giá trị nào sẽ được tối ưu hóa. Trình tối ưu hóa cũng có tham số tốc độ học (lr), xác định mức độ cập nhật sẽ được thực hiện ở mỗi bước. Các trình tối ưu hóa khác nhau cũng có các siêu tham số khác nhau.

```
import torch.optim as optim
```

Sau khi có hàm tối ưu hóa, chúng ta có thể định nghĩa một mất mát mà chúng ta muốn tối ưu hóa. Chúng ta có thể tự định nghĩa mất mát hoặc sử dụng một trong các hàm mất mát được định nghĩa trước trong PyTorch, chẳng hạn như nn.BCELoss(). Bây giờ chúng ta hãy ghép mọi thứ lại với nhau. Trong ví dụ này, chúng ta sẽ bắt đầu bằng cách tạo một số dữ liệu giả.

```
# tao y
y = torch.ones(10, 5)
# them nhieu vao y
x = y + torch.randn_like(y)
x
```

```
tensor([[ 1.4604,  2.1893,  0.8026,  0.6062,  0.9302],
        [ 0.2317,  1.1250,  1.1276,  0.3870,  2.0563],
        [ 0.3060,  2.9154,  0.0979,  0.8160,  1.9484],
        [ 0.2887,  0.1449,  0.1708, -0.0307,  2.0472],
        [ 0.6013,  1.8131,  0.7032,  0.2523, -0.0232],
        [ 2.6874,  0.5569,  0.4187,  1.2088,  0.2832],
        [ 2.8989,  1.5482,  0.3915,  0.5046,  0.1376],
        [ 1.1544,  0.3065, -0.2656,  2.5511,  0.2247],
        [ 1.7423, -0.1496,  2.0271,  1.0739,  1.1934],
        [ 0.3968,  0.2009,  0.6164,  1.8774,  1.1588]])
```

Tiếp theo, chúng ta có thể xác định mô hình, bộ tối ưu hóa và hàm mất mát.

```
# Khoi tao mo hinh
model = MultilayerPerceptron(5, 3)

# Dinh nghia ham toi u
adam = optim.Adam(model.parameters(), lr=1e-1)

# Dinh nghia ham loss function
loss_function = nn.MSELoss()

y_pred = model(x)
loss_function(y_pred, y).item()
```

Kết quả:

```
0.18392038345336914
```

Tiếp theo, chúng ta có thể thiết lập vòng lặp để huấn luyện mô hình:

```
n_epoch = 10

for epoch in range(n_epoch):
  adam.zero_grad()
  y_pred = model(x)
  # Get the loss
  loss = loss_function(y_pred, y)

  print(f"Epoch {epoch}: traing loss: {loss}")

  loss.backward()

  adam.step()
```

Kết quả:

```
Epoch 0: traing loss: 0.18392038345336914
Epoch 1: traing loss: 0.1214178055524826
Epoch 2: traing loss: 0.0665205642580986
Epoch 3: traing loss: 0.031827643513679504
Epoch 4: traing loss: 0.012266852892935276
Epoch 5: traing loss: 0.003626907477155328
Epoch 6: traing loss: 0.000923578510992229
Epoch 7: traing loss: 0.00023057256476022303
Epoch 8: traing loss: 6.007583942846395e-05
Epoch 9: traing loss: 1.6709067494957708e-05
```

Liệt kê bộ tham số học:

```
list(model.parameters())
```

Kết quả:

```
[Parameter containing:
 tensor([[-0.3774, -0.2663, -0.2254, -0.3110, -0.0272],
         [ 1.1547,  0.4672,  0.5807,  0.3853,  0.7197],
         [ 0.5740,  1.1962,  0.7437,  0.9969,  1.0613]], requires_grad=True),
 Parameter containing:
 tensor([0.0332, 0.8876, 0.4750], requires_grad=True),
 Parameter containing:
 tensor([[ 0.3789,  0.8695,  0.9322],
         [-0.0926,  0.9584,  0.9451],
         [ 0.1741,  0.5681,  0.5995],
         [-0.5748,  1.1098,  0.6488],
         [-0.0048,  1.2007,  1.2029]], requires_grad=True),
 Parameter containing:
 tensor([0.8713, 1.1602, 0.9818, 0.8432, 0.4917], requires_grad=True)]
```

Chúng ta có thể thấy hàm mất mát của chúng ta đang giảm. Bây giờ chúng ta hãy kiểm tra các dự đoán của mô hình và xem liệu chúng có gần với y ban đầu của chúng ta không, tức là tất cả đều là 1.

```
y_pred = model(x)
y_pred
```

```
tensor([[1.0000, 1.0000, 0.9994, 1.0000, 1.0000],
        [0.9999, 1.0000, 0.9984, 0.9998, 1.0000],
        [1.0000, 1.0000, 0.9996, 1.0000, 1.0000],
        [0.9980, 0.9989, 0.9884, 0.9975, 0.9995],
        [0.9990, 0.9994, 0.9925, 0.9985, 0.9998],
        [0.9999, 1.0000, 0.9987, 0.9999, 1.0000],
        [1.0000, 1.0000, 0.9991, 1.0000, 1.0000],
        [0.9995, 0.9997, 0.9952, 0.9993, 0.9999],
        [1.0000, 1.0000, 0.9991, 1.0000, 1.0000],
        [0.9997, 0.9998, 0.9963, 0.9994, 1.0000]], grad_fn=<SigmoidBackward0>)
```

Tạo dữ liệu test và tính toán trên dữ liệu này:

```
x2 = y + torch.randn_like(y)
y_pred = model(x2)
y_pred
```

Kết quả:

```
tensor([[1.0000, 1.0000, 0.9999, 1.0000, 1.0000],
        [0.9973, 0.9985, 0.9859, 0.9976, 0.9993],
        [1.0000, 1.0000, 0.9997, 1.0000, 1.0000],
        [0.9997, 0.9998, 0.9966, 0.9991, 1.0000],
        [1.0000, 1.0000, 0.9999, 1.0000, 1.0000],
        [0.9983, 0.9990, 0.9893, 0.9971, 0.9996],
        [1.0000, 1.0000, 0.9993, 1.0000, 1.0000],
        [1.0000, 1.0000, 0.9994, 1.0000, 1.0000],
        [0.9999, 1.0000, 0.9988, 0.9999, 1.0000],
        [0.9999, 0.9999, 0.9979, 0.9998, 1.0000]], grad_fn=<SigmoidBackward0>)
```

1.5 Bài tập

Bài 1

Giải thích quy trình lan truyền ngược trong mạng nơ-ron bằng PyTorch. Tại sao cần gọi loss.backward()

Bài 2

Viết code PyTorch để định nghĩa mạng nơ-ron 3 lớp (input=10, hidden=20, output=2) với:

- Hàm kích hoạt ReLU cho tầng ẩn
- Softmax cho đầu ra
- Loss function phù hợp cho bài toán phân loại

Bài 3

Liệt kê 3 kỹ thuật chống overfitting trong PyTorch và cách triển khai cụ thể. Cho ví dụ code sử dụng Dropout.

Bài 4

So sánh 2 optimizer trong PyTorch: SGD và Adam. Khi nào nên dùng mỗi loại? Viết code khởi tạo chúng với learning rate=0.01.

Bài 5

Mô tả cách tạo DataLoader trong PyTorch cho tập MNIST. Tại sao cần dùng Dataset và DataLoader?

Bài 6

Khi accuracy không cải thiện trong quá trình training, bạn sẽ kiểm tra những gì? Liệt kê 5 bước debug cơ bản.

Bài 7

Hãy vẽ mô hình thiết kế mạng nơ ron cho bài toán phân lớp văn bản (ví dụ phân tích sentiment). Đầu vào là một câu, đầu ra là phân lớp câu theo 3 nhãn positive, negative, neural.

Bài 8

Hãy vẽ mô hình thiết kế mạng nơ ron cho bài toán phân lớp ảnh (ví dụ như phân lớp chó/mèo hoặc phân loại số viết tay). Đầu vào là một ảnh, đầu ra ảnh thuộc lớp nào.

Bài 9

Với mô hình được thiết kế như **Bài 7**, hãy sử dụng pytorch định nghĩa mô hình, thực hiện huấn luyện trên bộ dữ liệu cụ thể, đánh giá mô hình.

Bài 10

Với mô hình được thiết kế như **Bài 8**, hãy sử dụng pytorch định nghĩa mô hình, thực hiện huấn luyện trên bộ dữ liệu cụ thể, đánh giá mô hình.